

Figures/Airbus_A350.jpg

Rover Navigation using NMPC methods

Francisco Miguel Carneiro de Castro

Thesis to obtain the Master of Science Degree in

Aerospace Engineering

Supervisor(s): Prof. Rodrigo Ventura

Examination Committee

Chairperson: Prof. Full Name

Supervisor: Prof. Full Name 1 (or 2)

Member of the Committee: Prof. Full Name 3

Month 2025

Dedicated to someone special...

Declaration

I declare that this document is an original work of my own authorship and that it fulfills all the requirements of the Code of Conduct and Good Practices of the Universidade de Lisboa.

Acknowledgments

A few words about the university, financial support, research advisor, dissertation readers, faculty or other professors, lab mates, other friends and family...

Resumo

Inserir o resumo em Português aqui com o máximo de 250 palavras e acompanhado de 4 a 6 palavras-chave...

Palavras-chave: palavra-chave1, palavra-chave2,...

Abstract

Insert your abstract here with a maximum of 250 words, followed by 4 to 6 keywords...

Keywords: keyword1, keyword2,...

Contents

Acknowledgments	vii
Resumo	ix
Abstract	xi
List of Tables	xvii
List of Figures	xix
Nomenclature	xxi
Acronyms	xxiii
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement	4
1.3 Project outline	4
1.4 Objectives	4
1.5 Contributions	4
1.6 Thesis Outline	4
2 State of the art	5
2.1 Path planning algorithms	5
2.1.1 World modelling	5
2.1.2 Path planning	6
3 Proposed Approach	7
3.1 Problem formulation	7
3.2 Optimal path planning	7
3.2.1 Cost map generation	7
3.3 Mobile robot kinematics	8
3.3.1 Wheel kinematics	9
3.3.2 Wheel-terrain contact	10
3.3.3 Slippage	12
3.4 Mobile robot dynamics	12
3.4.1 Wheel actuation	14
3.4.2 Wheel spin rate	15

3.4.3	Wheel-terrain contact model	15
3.5	Full state-space model	17
3.6	Model verification	18
3.6.1	Maximum inclination	18
4	Background	19
4.1	Two-layer map generation	19
4.2	Fast Marching Method	20
5	Global path planner	21
5.1	Traversability map computation	21
5.1.1	Height minimization	21
5.1.2	Maximum terrain roughness	23
5.1.3	Worst-case scenario	24
5.1.4	Implementation	25
5.1.5	Discussion	27
5.2	Optimal path planning	28
5.2.1	Implementation	29
5.2.2	Discussion	30
6	Control system	33
6.1	Problem Description	33
6.2	Baseline Solution	33
6.3	Enhanced Solution	33
6.3.1	Figures	33
6.3.2	Equations	34
6.3.3	Tables	35
6.3.4	Mixing	36
7	Conclusions	39
7.1	Achievements	39
7.2	Future Work	39
	Bibliography	41
A	Mathematical snippet	43
A.1	Euler angles	43
A.2	Skew-symmetric matrix	43
A.3	Inertia tensor	44
A.4	Differentiation of a vector	44
A.5	Parallel-axis theorem	44

B	Miscellaneous	45
B.1	Elevation maps	45
B.2	Traversability maps	45

List of Tables

1.1	Planetary, autonomous, wheeled rovers achievements.	2
1.2	Slippage-related hazards faced by rovers at planetary environments.	3
5.1	Table caption shown in TOC.	25
5.2	Elevation maps dimensions.	26
5.3	Table caption shown in TOC.	26
5.4	Comparison between algorithms 2 and 3 (Map 1)	28
5.5	Comparison between algorithms 4 and 5 (Map 1)	28
5.6	Algorithm 7 parameters.	30
6.1	Table caption shown in TOC.	35
6.2	Memory usage comparison (in MB).	36
6.3	Another table caption.	36
6.4	Yet another table caption.	36
6.5	Very wide table.	36

List of Figures

2.1	World models and map types	5
3.1	Car-like robot kinematics [14]	8
3.2	Wheel kinematics	10
3.3	Mobile robot systems. Adapted from [1]	13
3.4	Forces and moments distribution.	13
3.5	Wheel-terrain contact models [1]	15
5.1	Hull points on each wheel	26
5.2	Traversability maps (case 1; map 1)	27
5.3	Traversability maps (case 1; map 1)	27
5.4	Cost to speed conversion (equation 5.8)	29
5.5	Potential flow and speed map for worst-case traversability map (case 1; map 1)	31
5.6	Optimal paths (case 1; map 1)	31
5.7	Paths comparison and GETOPTIMALPATH(·) method runtimes.	32
5.8	Optimal path generation module.	32
6.1	Caption for figure in TOC.	33
6.2	Some aircrafts.	34
6.3	Schematic of some algorithm.	34
6.4	Figure and table side-by-side.	37
B.1	Elevation maps	45
B.2	Traversability maps (case 2; map 1)	46
B.3	Traversability maps (case 2; map 1)	46

Nomenclature

Greek symbols

ϕ, θ, ψ Roll, pitch, and yaw angles, respectively.

Roman symbols

χ Mobile robot orientation.

$\mathbf{p}$ Mobile robot position.

$\mathbf{q}$ Mobile robot pose.

$\mathbf{R}$ Rotation matrix.

$\mathbf{v}$ Mobile robot velocity.

$\mathcal{B}, \mathcal{W}$ Body and world frames, respectively.

Acronyms

3-D Three Dimensional. 6

casADi casADi (open-source software tool). 26

COM Center of mass. 8, 12–14, 22

DEM Digital Elevation Models. 5–7, 19

DOF Degree of freedom. 2, 8, 9, 19

EVA Extra-vehicular activity. 1

FMM Fast Marching Method. xiv, 20

GNC Guidance, Navigation and Control. 2

HHM Heterogeneous and Hierarchical Models. 5, 6

ICR Instantaneous center of rotation. 9

IPOPT Interior-point optimizer. 26

ISS International Space Station. 1

MER Mars Exploration Rover. 1

MSL Mars Science Laboratory. 1

NLP Non-linear programming. 22, 26–28

NMPC Nonlinear Model Predictive Control. 7

ODE Open Dynamics Engine. 15, 16

ROS Robot Operating System. 6

SA Semantic Attributes. 5, 6

Chapter 1

Introduction

Space robotics covers all robotic systems which operate off the Earth. Firstly, these systems may be classified according to its operational environment, whether it is planetary or orbital. On a second stage, a space robot is comprised of locomotion and autonomy features depending on its goal [?].

Space robot locomotion may take several approaches like manipulation, gripping, roving, drilling, sampling, flying, floating, diving or even climbing. On the other hand, a space robot's autonomy ranges from teleoperation to fully-autonomous operations, while other robots execute tasks on a semi-autonomous operation mode.

In 1981, the first orbital robot, Canadarm, was placed on orbital environment with the goal to move payloads from the Space Shuttle to a given deployment position [1]. Later in 2001, Canadarm 2 was launched in order to assist astronauts during EVA, but also to maintain and build the ISS. The humanoid robot Robonaut 2 was launched to the ISS in 2011 to gain experience while performing assisting or teleoperation tasks, for the sake of upgrading it to a future version in order to assist astronauts during EVA. The Moon sampler Surveyor 3 was the first planetary robot to be launched (1967). From the 90s onwards, Mars has become the main exploratory target for surface robots. The *Sojourner* rover, the twin Spirit and Opportunity MER and the MSL have provided valuable scientific information and have enabled ground teams to design rovers which reach higher velocities and larger lifetimes. Recently, in 2021, the Perseverance rover landed on Mars taking on its belly a small robotic helicopter which accomplished the first powered, controlled flight on the thin atmosphere of Mars [?].

Robots which operate without human intervention have a limited lifetime, which places a limit on the returned scientific profit. This thesis will focus on providing a navigation method which aims at delivering a short and safe path while going through in a timely fashion an semi-unknown location.

1.1 Motivation

A conventional planetary, autonomous rover typically assumes a 6-wheeled articulated configuration proposed by Bekker in the 60s, after comparing several designs including tracked or screw-type vehicles [? ?]. This configuration fosters crossing of crevasses, rocks or steps up to three times the radius of

the wheel, while generating greater stability, redundancy, continuous wheel contact with the terrain to enforce tractability and better distribution of the vehicle load [?].

Although this thesis will not focus on robot design, but rather on delivering a proposal for its GNC system, it is worth mentioning alternative configurations. Additional DOF and active suspensions on a 6-wheeled configuration (for instance the rocker-bogie system [? ?]) to counteract obstacles in an energy-efficient manner, is an example extensively discussed in the literature. Screw-type vehicles [?] like helical geometries and blades are a useful configuration to generate thrust on unconsolidated or amphibious environments, where wheels cannot react against the terrain in order to generate thrust. Alternative configurations like climbing robots have been on experimental stage for space as well. The use of microspines [?] by reacting to passive pulling away from a non-horizontal surface is a recent development on climbing robots research. Repelling robots which rely on a tether as an anchor is a proposed method as a way to perform subsurface exploration [?].

As it is stated in Table 1.1, launched planetary, autonomous rovers have achieved higher lifetimes as well as higher traverse speeds and travelled greater distances each mission goes by. Below, only rovers deployed in Mars are shown, but the CNSA has also deployed two rovers on the Moon, - *Yutu and Yutu-2* - of which little information is known.

Rover	Mass (kg)	Lifetime (sol)	Distance travelled (km)	Max. traverse speed (cm/s)	Ref.
<i>Sojourner</i>	10	83	0.106	0.6	[? ?]
<i>Spirit</i>	185	2623	7.73	1.0	[? ? ?]
<i>Opportunity</i>	185	5352	45.16	1.0	[? ? ?]
<i>Curiosity</i> *	899	4068	31.15	2.5	[? ? ?]
<i>Perseverance</i> *	1025	1021	23.776	4.2	[?]
<i>Zhurong</i> **	240	347	1.921	n.a.	[?]

* Still in operation (as of 16 January 2024)

** Inactive (as of 20 May 2022)

Table 1.1: Planetary, autonomous, wheeled rovers achievements.

Although deployed rovers have achieved greater coverages, these have also faced slippage-hazardous situations [?] and are subject to get around uncrossable obstacles or stay away from wind devils. These may damage the rover systems, cause progress delay or determine the end of the mission.

Slippage-related hazards may play an important role on preventing a rover to achieve its proposed goals. Even though some slip is desirable to generate traction, excessive slip will result in loss of traction [?] and possibly entrap the rover. These events are directly linked to the type of wheel-soil interaction taking place [?]. The nature of this interaction takes into account both the type of wheel (rigid or compliant) and its features as well as the type of soil (firm or deformable) and its properties. Apart from it, other variables weighing into slip are the vertical load acting on the soil by the wheel and sinkage, which will cause a decrease on effective thrust and increase drawbar pull [? ?]. In this thesis, only rigid wheels are used. These face higher slips on deformable ground, due to high sinkage and shear displacement, while slip is tremendously reduced in the case of compact soils [?]. Due to the impossibility of mimicing deformable ground dynamics on real-time simulation test-bed, only rigid

terrains are considered. In Table 1.2, one can find slippage events faced by wheeled, autonomous rovers in planetary environment as well as the teleoperating actions put in place to attempt a reversal from the state the rover was at.

Rover	Date of Event	Terrain, Place	Hazard	Action
Curiosity	May, 2015	Deformable terrain (Mars, Gale Crater, near Mount Sharp)	High-slip events	Turn back the rover before it could become entrapped
Curiosity	February, 2014	Deformable terrain (Mars, Gale Crater, near Mount Sharp)	High-slip events (sinkage up to 0.15 [m] and slippage up to 77%)	The route was manually biased to avoid soft sands
Opportunity	April 6, 2010	High windblown ripple dominated by loose, poorly sorted basaltic sands (Mars, Meridiani Planum)	Opportunity experienced significant slippage (up to 40%)	More power to the wheels and extra slip checks
Spirit	May 1, 2009	Soft soil (Mars, Gusev Crater, Home Plate)	Spirit got entrapped with no possibility of recovery	Several recovery techniques (mission concluded on May 24, 2011)
Opportunity	April 26, 2005	Sand dune (Mars, Purgatory Ripple, Meridiani Planum)	Opportunity got entrapped	Recovery actions (for 5 weeks)
Sojourner	August 21, 1997	Deformable terrain (Mars, Ares Vallis)	The right front wheel was parked on a small rock, perching the left front wheel above the surface	Engineers teleoperated the rover to a safer place

Table 1.2: Slippage-related hazards faced by rovers at planetary environments [?].

Very small velocities have been achieved by planetary rovers until this point. Apart from lunar teleoperated/driving vehicles, wheeled, autonomous robots have not experienced a dynamic terramechanics regime. In [?], a technical review on strategies to increase the operational velocity is presented. In this thesis, a wheeled, autonomous robot is considered to be on high-speed, if it approaches or surpasses 1 m/s, which is roughly ten times larger than the maximum traverse rover velocities presented in Table 1.1.

The Bekker method [? ? ?] is capable of providing a definition of the stress fields developed as a result of the interaction between the terrain and the vehicle wheels. However, this method is only accurate on a quasistatic regime, as it fails to accurately quantify soil behaviour in a dynamic regime. Pope stated that the rate of penetration plays a role on determining the normal stress distribution applied on the wheel [?]. Therefore, not only the slip ratio but also the vehicle's absolute speed is a factor, when it comes to determining both its sinkage and rolling resistance under an irregular terrain. In 1971, Pope proved that by increasing the vehicle velocity, it is also possible to reduce rolling resistance and its sinkage [?]. In 1991, Grahm proposed a model which takes the Bekker pressure-sinkage relationship as its basis, but extends the formulation to include a dynamic regime (considered by Grahm to happen above 1 cm/s), by relating the rate of penetration of a rigid wheel with the wheel driving speed and slip ratio [?]. From Grahm's model, it can be inferred that, for a given load, the maximum sinkage decreases with increasing traveling velocities. Additional details on the mentioned models and empirical evidence from planetary missions which involved high-speed vehicles can be found in [?].

Secondly, slippage compensation strategies [?] shall be included as a safety layer of a navigation control architecture in order to avoid entrapment or embedding situations. These solutions range from torque-based compensation solutions to velocity-based solutions. Torque-based solutions include max-

imizing traction while ensuring the soil never fails or limiting the torque control signal such that the desired slip matches the estimated slip. These methods - also called motor-current slippage compensation methods - rely on complex terramechanics models, which require the measure of several parameters in real-time, thus limiting the robot's velocity. Conversely, velocity-based slippage compensation methods do not require estimation of parameters from terramechanics models, though these are expected not to perform well in challenging terrain like slopes or sand ripples. This latter approach may include solutions which aim at resolving the so-called kinematic incompatibility problem [?] - "wheel fighting" as result of wheel slip on irregular, off-the-road terrains -, adapt control signals with respect to the estimated slip and limit the velocity and acceleration of wheels although this latter approach may result in non-controllable slip hazards because the robot will slip even with a limited motion domain.

As a result of tackling these issues, it is expected to maximize the operational timeline, by decreasing the time dedicated to go through the planetary environment to reach locations of scientific interest and increase the available time for scientific operations. Extending the covered surface is also a desirable navigational outcome, which may be eased by providing the robot with a prior knowledge of the scouting area [? ?]. This thesis will focus on providing a navigational control architecture which addresses the issues in the diagnosis presented previously, by taking safety, fastness and covered area as the main metrics of success.

1.2 Problem Statement

1.3 Project outline

1.4 Objectives

1.5 Contributions

1.6 Thesis Outline

Chapter 2

State of the art

2.1 Path planning algorithms

In this section, a review on the generation of paths for mobile robot tracking is presented. Moreover, a review on the modelling of maps for optimal path planning is also presented, based on the review of Burgard et al. [2].

2.1.1 World modelling

World modelling is a *must* when it comes to optimally compute a path for a mobile robot to follow. Due to that, optimal decisions may be taken by a system at any configuration. World modelling falls into the following categories (figure 2.1): indoor and structured environments, natural terrains or dynamic environments. In this thesis, one is interested in performing a metrical evaluation of a mars-like environment with the absence of dynamic obstacles, so a brief review of models for natural terrains is presented.

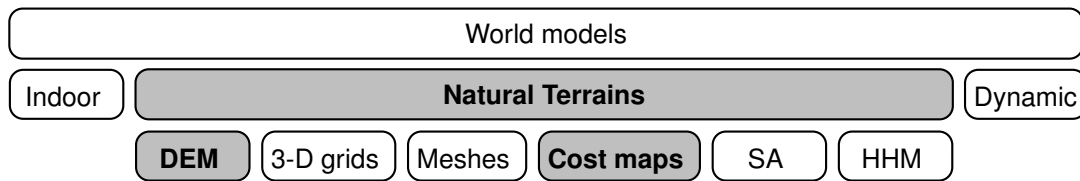


Figure 2.1: World models and map types

Elevation grids or **digital elevation maps** (DEM) are geometric models which store the elevation h of a discrete location (x_i, y_j) . Elevation grids are generated due to sensor data. This retrieval of data may be aerial or on the ground. In case of aerial retrieved data, a regularly sampled grid is typically the end result, due to uniform sensor data. If data is retrieved from the ground, the quality of the data depends on the incidence angle and the distance of the sensor to the location (x_i, y_j) , which typically yields a variable-sized grid. In the work of Fankhauser et al. [3], a robot-centric, online updated elevation map is estimated, where each cell stores a height estimate $\hat{h}$ and a variance $\hat{\sigma}_h^2$. From a wheeled, mobile planetary exploration point of view, in [4] a robot-centered DEM is built from a 3D point cloud, which was generated by a pair of stereo cameras. Aerial data [5] is also used to deliver an *a-priori* global grid of

the target environment, with an accuracy that can be improved by a ground, robot-centered mapping system [4, 6]. Additionally, information on how to tackle issues such as sensor resolution, uncertainty and terrain occlusion can be found in [2].

3-D grids and point sets extend the previous approach to a 3-D representation. Typically, this approach handle large sets of points where free or occupied areas can't be defined directly. Octotrees are a solution to this problem. Then, the Octomap approach [6] uses octotrees to support compact memory representation, multiresolution queries and probabilistic occupancy estimates. A great advantage is that it is possible to model areas which are occluded by vegetation and assign occupancy costs to them.

Meshes present an improvement to Digital Elevation Models and 3-D grids. It is a compact representation and presents continuity over the surface. However, this method requires simplifications techniques to lower computational costs and presents some difficulty on defining complex environments as a continuous surface, such as vegetation. Therefore, precise data is needed to detect discontinuities.

Cost maps are a feasible way to model traversability costs of a rough, uneven terrain and offer a safe and well-known approach to guidance systems on such kind of environments. **Cost maps** may be updated *in-situ* or generated *prior* to placement with overhead imagery. Usually, the computation of costs takes into account the robot kinematics [7] and dynamics [8], as well as terrain inclination. The definition of an explicit cost function is arbitrary, although the robot's limitations are a factor to include. At the end, the obtained traversability map can be combined with a minimum-cost planner [7, 9], in order to output optimal paths into a control system. In the work of Amorim and Ventura [7], a traversability cost is generated at each point by minimizing the height of a mobile robot at each location (x_i, y_j) . That minimization is carried out by fixing the center of mass of the robot at (x_i, y_j) , and then the mobile robot is rotated until the lowest admissible height is achieved. In the work of Ishigami et al. [9], a criteria function for the **Dijkstra's** algorithm is described, by assigning to each cell a cost, which takes into account the terrain roughness around that cell, distance to goal and terrain inclination. In [10], a **cost map** is created for an occupancy grid, by making obstacle space impossible to traverse and close to obstacle cells progressively less costly to traverse until a safety distance is reached.

Semantic attributes (SA) provide information about the type of terrain and objects found. This representation is called a *high-level* one, in the sense that it groups data into broader classes of data, such as traversable or occupied regions, terrain type or vegetation. On the other hand, the previous approaches are defined as *low-level*. Terrain classification methods [11] are used to determine the type of terrain and landmark extraction is useful to detect saliencies in the environment.

Finally, **Heterogeneous and hierarchical models (HHM)** offer hybrid maps which combine metric and feature maps. In [12], a C++ library with a Robot Operating System (ROS) and OpenCV interfaces is described. It manages a multi-layer map for rough terrain navigation. It is designed to store data such as elevation, traversability and surface normals.

2.1.2 Path planning

Chapter 3

Proposed Approach

3.1 Problem formulation

The problem laid out in this thesis is to yield a Nonlinear Model Predictive Control (NMPC) [? ?] algorithm $\mathbb{P}$, which models the kinematics and dynamics of a rigid wheeled, autonomous robot $\mathcal{R}$ as well as its interactions with the terrain during the traversal of a semi-unknown, off-the-road, uneven terrain map $\mathcal{M} \subset \mathbb{R}^3$, resembling a planetary environment. It is required that the robot avoids untraversable areas or obstacles $\mathcal{O} \subset \mathcal{M}$, and follows a time optimal traversable pathway to reach a goal point $G \in \mathcal{M}$, starting from a given initial admissible pose $\mathcal{R}_p \in \mathcal{M} \setminus \mathcal{O}$. Moreover, the robot $\mathcal{R}$ shall make a fast, safe and slippage-free traverse, as well as minimizing the energy consumed by the robot battery. The algorithm $\mathbb{P}$ must be computationally optimal and update the system actuation in a timely fashion.

3.2 Optimal path planning

3.2.1 Cost map generation

This thesis is mainly focused on Nonlinear Model Predictive Control, so a simple approach to yield a suitable traversability map is chosen. Both the elevation, $\mathcal{E}$, and traversability, $\mathcal{T}$, maps are assumed to be known *a-priori* and will be computed off-line.

Assumption 1. *The elevation and traversability maps are assumed to be known a-priori without uncertainties, resembling the fact that planetary terrains are scanned prior to the placement of a rover on the ground.*

Due to its compactness and simplicity, Digital Elevation Models are chosen to describe the elevation h at each location (x_i, y_j) . Then, from the compacted data, a cost map approach is followed in order to yield a traversability map of the environment, by assigning a traversability cost c to each grid point (x_i, y_j) . This choice stems from the fact that this is a popular trend in the planetary robotics literature, but also because it is easily combined with a minimum-cost planner algorithm, in order to output a global planner. Finally, the elevation and traversability maps can be joined in a two-layered map $\mathcal{M}_2$, because

both domains are equal. Moreover, these frames are considered to share the same frame $\mathcal{W}$, because both map frames match at the origin. An algorithm to address this plan is presented in section 5.1. It is built on top of the works of Amorim and Ventura [7] and Ishigami et al. [9] and provides [complete subsection by linking map generation to path planning algorithm and by adding short description of the implementation.](#)

3.3 Mobile robot kinematics

Kinematics describe the motion geometry of a point or body according to the provided actuations. A four-wheeled rover-like kinematics model is to be derived in this chapter, which can be applied both to skid-steering or differentially driven mobile robots.

A wheeled mobile robot $\mathcal{R}$ is moving on a map $\mathcal{M}$, which is centered at the origin of an inertial frame $\mathcal{W}$. A moving frame $\mathcal{B}$ is centered on the robot's center of mass COM and it is fixed to $\mathcal{R}$. Although the mobile robot operates on unstructured, uneven, off-the-road terrains, not all DOF are controllable. This issue is addressed in the following sections.

Assumption 2. In moving frame $\mathcal{B}$, $\mathcal{B}_x$ points forward, $\mathcal{B}_y$ points to the left and $\mathcal{B}_z$ points upwards. Regarding world frame $\mathcal{W}$, its axis directions are dictated by ROS' standard axis directions [13].

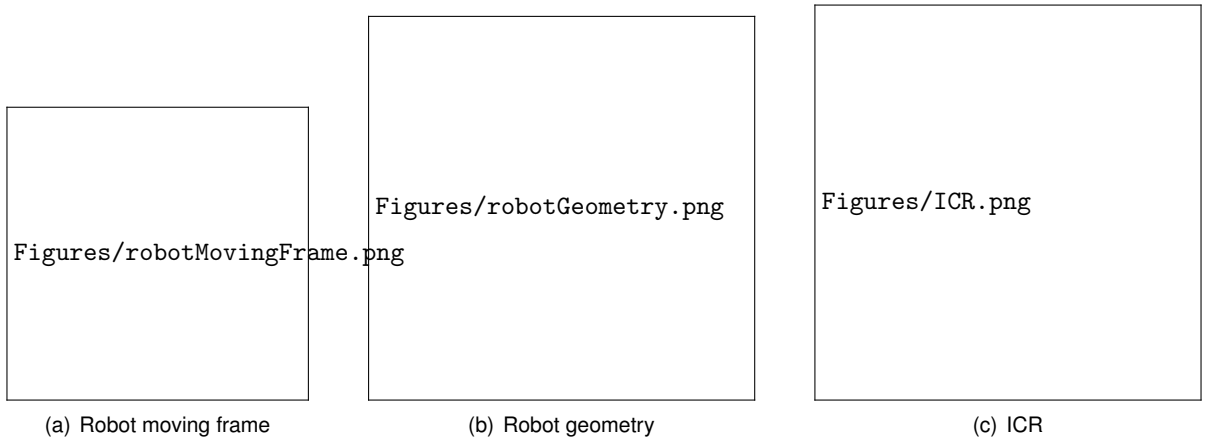


Figure 3.1: Car-like robot kinematics [14]

The mobile robot position and orientation with respect to frame $\mathcal{W}$ is described as $\mathbf{p} = [x \ y \ z]^T$ and $\Psi = [\phi \ \theta \ \psi]^T$, respectively. On the other hand, the mobile robot linear and angular velocities with respect to frame $\mathcal{B}$ stands as $\mathbf{v} = [u \ v \ w]^T$ and $\boldsymbol{\omega} = [p \ q \ r]^T$, respectively. Then, it is possible to map the mobile robot $\mathcal{R}$ linear velocity between frames $\mathcal{B}$ and $\mathcal{W}$ by taking advantage of rotation matrix A.2, but also to map euler angle rates in $\mathcal{W}$ to angular velocities in $\mathcal{B}$ with rotation matrix ??.

$$\dot{\mathbf{p}} = \mathbf{R}_B^{\mathcal{W}} \mathbf{v} \quad (3.1)$$

$$\dot{\Psi} = \bar{\mathbf{R}}_B^{\mathcal{W}} \boldsymbol{\omega} \quad (3.2)$$

3.3.1 Wheel kinematics

A kinematics model is not only described by its free-body motion equations 3.1 and 3.2, but also by geometrical constraints [14]. Each wheel is actuated by a torque τ_i , which induces on it a spin rate, ω_i , $i = 1, \dots, N_w$, where $N_w = 4$ for the cases presented in this thesis.

Assumption 3. *The thickness and deformation of each wheel are neglected so that the wheel is assumed to be rigid and to contact the terrain at a single contact point c_i , in case rigid terrain is the working environment [15].*

prove controlability when full model is presented Due to the fact that wheels are fixed, the mobile robot holds two controllable DOF: translation along its longitudinal direction $\mathcal{B}_x$, and rotation around its vertical axis $\mathcal{B}_z$. Therefore, lateral skidding from the vehicle is expected to remain close to zero on flat terrain, although its wheels are prone to skidding when the mobile robot finds itself on uneven or inclined surfaces (figure 3.4). If the mobile robot follows a straight line path, its wheel velocities are tangent to the current path. Moreover, the velocity vector of each wheel is tangent to the terrain that wheel is touching on.

The terrain profile imposes p and q . On the other hand, the existence of holes or bumps force w to be different from zero. v rises above zero in case of lateral skidding.

If a mobile robot is not following a straight line path, this robot will rotate around an ICR point. Each wheel angular velocity is equal to the mobile robot angular velocity, ω . The vector $\mathbf{d}_i$, which connects the ICR point to each wheel position, is orthogonal to each wheel velocity $\mathbf{v}_i = [u \ v \ w]^T_i$, as well as $\mathbf{d}_C \perp \mathbf{v}$, so the following relations may be derived by taking into account the general expression $\mathbf{v} = \omega \times \mathbf{r}$. Vectors $\mathbf{v}_i$, $\mathbf{d}_i$, $\mathbf{d}_C$ and $\mathbf{r}_i$ are all expressed with respect to frame $\mathcal{B}$.

$$\begin{cases} u = qd_{C,z} - rd_{C,y} & u_i = qd_{i,z} - rd_{i,y} \\ v = rd_{C,x} - pd_{C,z} & v_i = rd_{i,x} - pd_{i,z} \\ w = pd_{C,y} - qd_{C,x} & w_i = pd_{i,y} - qd_{i,x} \end{cases} \quad (3.3)$$

Then, it is possible to relate $\mathbf{d}_C$ to $\mathbf{d}_i$, with $\mathbf{d}_C = \mathbf{d}_i + \mathbf{r}_i$, where $\mathbf{r}_i$ is a vector which connects each wheel center's position to the moving frame origin. If this relationship is introduced into equation 3.3, it comes that,

$$\mathbf{v}_i = \mathbf{v} + \omega \times \mathbf{r}_i \quad (3.4)$$

From the analysis of the previous equation, it is possible to state that,

$$\begin{cases} u_1 = u_2 \\ u_3 = u_4 \end{cases}, \quad \begin{cases} v_1 = v_4 \\ v_2 = v_3 \end{cases}, \quad \begin{cases} w_3 - w_2 = (r_{3,y} + r_{2,y})p \\ w_4 - w_1 = (r_{4,y} + r_{1,y})p \\ w_2 - w_1 = (r_{2,x} + r_{1,x})q \\ w_3 - w_4 = (r_{3,x} + r_{4,x})q \end{cases} \quad (3.5)$$

3.3.2 Wheel-terrain contact

In this thesis, it is important to retrieve each wheel's contact point with the terrain it is touching on, in order to enable the transfer of forces applied at c_i to frame $\mathcal{B}$ and to describe the velocity of each wheel with respect to a frame $\mathcal{T}_i$.

Assumption 4. Every wheel is touching the terrain at a point c_i .

Lemma 1. The velocity of each wheel is tangent to the terrain profile at c_i .

Assumptions 4 and 1 ensure this analysis remains consistent with the fact that, without a contact point c_i between a wheel and the terrain, there would be no reason to obtain a contact angle or a wheel velocity tangent to the terrain.

At every wheel i , a frame $\mathcal{T}_i$ is attached to the wheel center and follows up the terrain profile slope longitudinally. In accordance with assumption 3, roll effect on the determination of c_i is neglected. The transformation from $\mathcal{T}_i$ to $\mathcal{B}$ is based on a rotation matrix expressed in equation 3.6. Frame $\mathcal{T}^i$ cannot yaw with respect to $\mathcal{B}$ because the wheels do not steer ($\psi_i = 0$) and the lateral component, $\mathcal{T}_{y,i}$, matches with the wheel axis of rotation ($\phi_i = 0$).

$$\mathbf{R}_{\mathcal{T}^i}^{\mathcal{B}} = \begin{bmatrix} \cos(\theta_i) & 0 & \sin(\theta_i) \\ 0 & 1 & 0 \\ -\sin(\theta_i) & 0 & \cos(\theta_i) \end{bmatrix} \quad (3.6)$$

, where θ_i denotes the pitch component of the contact angle. Finally, it is possible to express $\mathbf{v}_i$ with respect to frame $\mathcal{T}^i$ (equation 3.7). This reasoning is depicted in figure 3.2.

$$\mathbf{v}_i = \mathbf{R}_{\mathcal{T}^i}^{\mathcal{B}} \mathbf{v}_i^{\mathcal{T}^i} \quad (3.7)$$

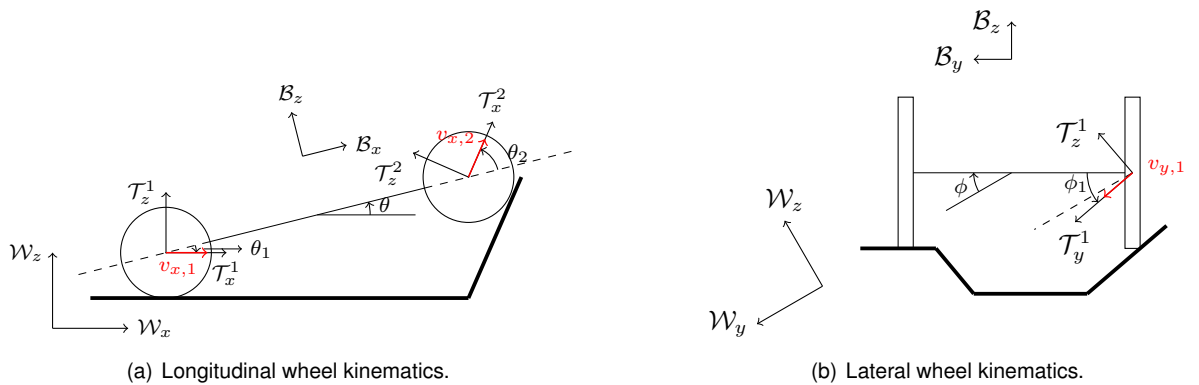


Figure 3.2: Wheel kinematics

Contact angle determination

Contact angle determination is of the utmost importance in case the mobile robot is crossing irregular terrains, because the traction forces are not aligned with $\mathcal{B}_x$. A method to compute θ_i is extracted from

[16]. In this thesis, only the equations to determine θ_i are presented. Further explanations can be found in [16], although it is worth mentioning that special cases are not included here.

Contact points

After obtaining the contact angles, it is possible to deliver each wheel-terrain contact point, c_i , coordinates with respect to $\mathcal{W}$ or $\mathcal{B}$.

At frame $\mathcal{T}_i$, $\mathbf{c}_i = [0, r \sin(\phi_i), -r \cos(\phi_i)]^T$. Therefore, it comes that,

$$(\mathbf{c}_i)_{\mathcal{W}} = \mathbf{p} + \mathbf{R}_{\mathcal{B}}^{\mathcal{W}} (\mathbf{r}_i + \mathbf{R}_{\mathcal{T}_i}^{\mathcal{B}} \mathbf{c}_i) \quad (3.8)$$

, where $\mathbf{R}_{\mathcal{T}_i}^{\mathcal{B}} = (\mathbf{R}_{\mathcal{B}}^{\mathcal{W}})^T \mathbf{R}_{\mathcal{T}_i}^{\mathcal{W}}$.

The first kinematic constraint (equation 3.9) states that right wheels have the same longitudinal velocity and left wheels also have the same longitudinal velocity, in frame $\mathcal{B}$. The second kinematic constraint (equation 3.10) states that front wheels have the same lateral velocity and back wheels also have the same lateral velocity, in frame $\mathcal{B}$.

$$\begin{cases} (u_1)_{\mathcal{T}} \cos(\theta_1) + (v_1)_{\mathcal{T}} \sin(\theta_1) \sin(\phi_1) = (u_2)_{\mathcal{T}} \cos(\theta_2) + (v_2)_{\mathcal{T}} \sin(\theta_2) \sin(\phi_2) \\ (u_3)_{\mathcal{T}} \cos(\theta_3) + (v_3)_{\mathcal{T}} \sin(\theta_3) \sin(\phi_3) = (u_4)_{\mathcal{T}} \cos(\theta_4) + (v_4)_{\mathcal{T}} \sin(\theta_4) \sin(\phi_4) \end{cases} \quad (3.9)$$

$$\begin{cases} (v_1)_{\mathcal{T}} \cos(\phi_1) = (v_4)_{\mathcal{T}} \cos(\phi_4) \\ (v_2)_{\mathcal{T}} \cos(\phi_2) = (v_3)_{\mathcal{T}} \cos(\phi_3) \end{cases} \quad (3.10)$$

The third constraint states that the difference between vertical velocities of left wheels (or right wheels) in frame $\mathcal{B}$ is equal the vehicle pitch rate times the longitudinal distance between these wheels. Lastly, the fourth constraint states that the difference between vertical velocities of back wheels (or front wheels) in frame $\mathcal{B}$ is equal the vehicle roll rate times the lateral distance between these wheels.

$$\begin{cases} -(u_1)_{\mathcal{T}}^{\mathcal{T}^1} \sin(\theta_1) + (v_1)_{\mathcal{T}}^{\mathcal{T}^1} \cos(\theta_1) \sin(\phi_1) + (u_2)_{\mathcal{T}}^{\mathcal{T}^2} \sin(\theta_2) - (v_2)_{\mathcal{T}}^{\mathcal{T}^2} \cos(\theta_2) \sin(\phi_2) = (a+b)q \\ -(u_4)_{\mathcal{T}}^{\mathcal{T}^4} \sin(\theta_4) + (v_4)_{\mathcal{T}}^{\mathcal{T}^4} \cos(\theta_4) \sin(\phi_4) + (u_3)_{\mathcal{T}}^{\mathcal{T}^3} \sin(\theta_3) - (v_3)_{\mathcal{T}}^{\mathcal{T}^3} \cos(\theta_3) \sin(\phi_3) = (a+b)q \end{cases} \quad (3.11)$$

$$\begin{cases} -(u_1)_{\mathcal{T}} \sin(\theta_1) + (v_1)_{\mathcal{T}} \cos(\theta_1) \sin(\phi_1) + (u_4)_{\mathcal{T}} \sin(\theta_4) - (v_4)_{\mathcal{T}} \cos(\theta_4) \sin(\phi_4) = 2cp \\ -(u_2)_{\mathcal{T}} \sin(\theta_2) + (v_2)_{\mathcal{T}} \cos(\theta_2) \sin(\phi_2) + (u_3)_{\mathcal{T}} \sin(\theta_3) - (v_3)_{\mathcal{T}} \cos(\theta_3) \sin(\phi_3) = 2cp \end{cases} \quad (3.12)$$

Provided that p , q , $(u_i)_{\mathcal{T}_i}$ and $(v_i)_{\mathcal{T}_i}$ are known, the contact angles ϕ_i

Simulate robot first without contact angle (assume plane of contacts is parallel to the same plane of the robot; then, include this estimation into the simulation; at rest (for maps generation), use gradients

Don't forget to include the pure rolling condition; it is violated when there is slip

3.3.3 Slippage

During the traversal of a planetary environment, a wheeled mobile robot may experience slip at each wheel i , both on the longitudinal (s_i) and lateral (β_i) directions of the wheel frame $\mathcal{T}_i$, where s_i stands for the slip ratio at wheel i and β_i denotes the slip angle at wheel i . Although the slip angle β_i may be measured at each wheel frame $\mathcal{T}_i$, it is more usual to do it in case of steering wheels. Instead, the slip angle is computed with the mobile robot COM velocities within frame $\mathcal{B}$.

$$s_i = \begin{cases} 1 - \frac{(u_i)_{\mathcal{T}_i}}{r \omega_i}, & \text{if } |r \omega_i| \geq |(u_i)_{\mathcal{T}_i}| \text{ (slip)} \\ \frac{r \omega_i}{(u_i)_{\mathcal{T}_i}} - 1, & \text{if } |r \omega_i| < |(u_i)_{\mathcal{T}_i}| \text{ (skid)} \end{cases} \quad \beta = \begin{cases} \arctan\left(\frac{v}{u}\right), & (u \neq 0) \\ \text{sgn}(v) \cdot \frac{\pi}{2}, & (u = 0) \end{cases} \quad (3.13)$$

, where r denotes the wheel radius. Physical parameters related to every wheel are equal. If $0 \leq s_i \leq 1$, it is said wheel i is slipping. If $-1 \leq s_i < 0$, it is said that wheel i is skidding or braking. However, if the signs of $(u_i)_{\mathcal{T}_i}$ and $\dot{q}_i$ are opposite to each other, it means $1 < |s_i| \leq 2$. Although these impractical slips are possible [17], no terramechanics analytical models are presented, because no literature was found with research on the topic. Regarding the slip angle β , this quantity is as high as the lateral skid, v , is with respect to the longitudinal velocity, u .

In accordance to the definitions presented above, it is possible to define a new quantity, δ_i , which expresses the relative velocity between wheel i at c_i and the surface.

$$\delta_i = r\omega_i - (u_i)_{\mathcal{T}_i} \quad (3.14)$$

In case $\delta_i = 0$, wheel i presents a pure rolling motion. Apart from that, wheel i exhibits a slipping or skidding behaviour.

Lemma 2. *Pure rolling condition is respected in case equation 3.14 equals zero.*

3.4 Mobile robot dynamics

A dynamics model describes the motion of a point or body subject to external forces. These forces may include gravity, wind blows, wheel-terrain traction or external agents disturbances.

On a mobile robot real-life application, a dynamics model is mainly triggered by gravity and wheel torques, which are actuated by electrical currents. A velocity change is to be expected, which will in turn yield an updated pose (figure 3.3).

The presented wheeled mobile robot model contains four controllable joints at each wheel COM, which are located at each wheel's center. The mobile robot may roll, pitch or fall vertically with respect to frame $\mathcal{B}$, because it will be moving through irregular terrain, which changes the balance of forces the mobile robot is subject to. Moreover, external forces (e.g. friction contact, wheel-terrain traction) and gravity are also taken into account in this general model. Typically, wind force is also an external force which acts on the mobile robot body at outdoor environments, but its effect was not introduced in this

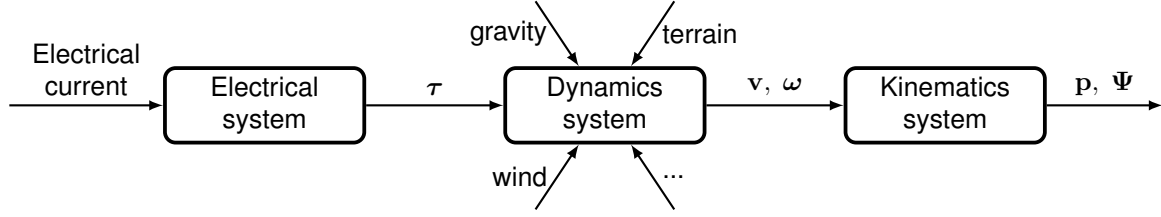


Figure 3.3: Mobile robot systems. Adapted from [1]

model.

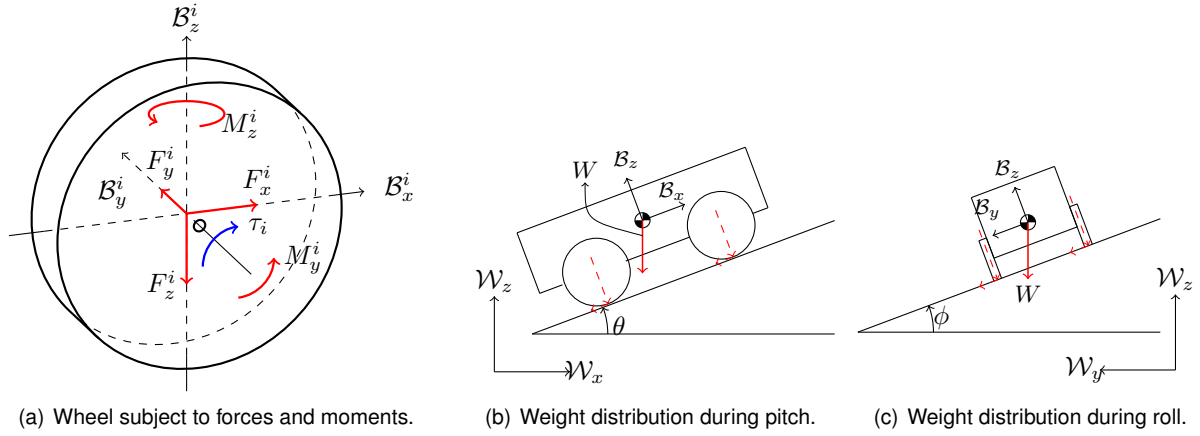


Figure 3.4: Forces and moments distribution.

Each wheel joint can only be actuated around the wheel horizontal axis, which is parallel to B_y . Every wheel holds a frame B_i parallel to B , centered at its COM. Then, it also holds a frame T_i which follows up the wheel velocity vector (section 3.3.2). Apart from it, each wheel is subject to forces and moments coming from its interaction with the terrain or due to gravity (figure 3.4). Finally, these forces and moments shall be expressed with respect to frame B and summed up in order to feed the dynamics relationship of the mobile robot (equation 3.15).

The weight distribution on the mobile robot wheels is influenced by its roll and pitch. A dynamics model is derived based on Newton's second law as well as on the angular momentum rate of change from each wheel around its respective COM. The angular momentum rate of change of the whole mobile robot around its COM is also considered.

$$\begin{aligned} m\dot{\mathbf{v}} + m\boldsymbol{\omega} \times \mathbf{v} &= \mathbf{f} \\ \mathbf{I}\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (\mathbf{I}\boldsymbol{\omega}) &= \mathbf{m} \end{aligned} \quad (3.15)$$

, where m stands for the mobile robot mass. Moreover, $\mathbf{I}$ denotes the inertia tensor of the mobile robot with respect to frame B . Furthermore, $\mathbf{f} = [f_x \ f_y \ f_z]^T$ stands for the sum of forces acting on the system and $\mathbf{m} = [l \ m \ n]^T$ denotes the sum of moments. $\mathbf{f}$ and $\mathbf{m}$ are expressed with respect to frame B .

Equation 3.15 may be obtained by applying the second newton law for translational and rotational dynamics. It is of interest to find out a dynamics formulation which is expressed with respect to B , because

$\mathbf{I}$ is constant there, but also because it is more intuitive to think of velocities, forces and moments with respect to $\mathcal{B}$. In case the linear and angular moments are defined by $\mathbf{l} = m\mathbf{v}$ and $\mathbf{h} = \mathbf{I}\boldsymbol{\omega}$ respectively, it is possible to derive a dynamics formulation with the help of equation A.7. From now on, for the sake of completeness, each moment and force acting on the mobile robot are presented.

3.4.1 Wheel actuation

Every wheel is commanded by a torque, τ_i , which is expressed with respect to $\mathcal{B}$ and it only holds a component in $\mathcal{B}_y$. That torque yields a traction force $\mathbf{t}_i$ which is expressed with respect to the wheel frame $\mathcal{T}_i$ (figure 3.4). The traction force $\mathbf{t}_i$ only holds a single longitudinal component within $\mathcal{T}^i$. $\mathbf{t}_i$ is the result of the action of the soil on the wheel at c_i . Therefore, the force produced by τ_i on the soil opposes $\mathbf{t}_i$. The wheel frame $\mathcal{T}_i$ is centered at the wheel center and follows up the wheel velocity vector, which may not match frame $\mathcal{B}_i$. Finally, it comes that,

$$\mathbf{t}_i = \begin{cases} \frac{\tau_i}{r}, & \left| \frac{\tau_i}{r} \right| - \mu(f_n)_i < 0 \\ \mu(f_n)_i \text{sign}(\tau_i), & \left| \frac{\tau_i}{r} \right| - \mu(f_n)_i \geq 0 \end{cases} \quad (3.16)$$

, where $\mathbf{w}_i$ stands for a vector which connects each wheel center to c_i , with respect to $\mathcal{B}$. Therefore, in order to determine the total force and moment produced by wheel actuation, one needs to express every traction force $\mathbf{t}_i$ with respect to $\mathcal{B}$, by taking into account equation 3.6. Then, the sum of traction forces, $\mathbf{t}$, with respect to $\mathcal{B}$ are expressed as,

$$\mathbf{t} = \sum_{i=1}^4 \mathbf{R}_{\mathcal{T}_i}^{\mathcal{B}} \mathbf{t}_i \quad (3.17)$$

Assumption 5. Moments produced around $\mathcal{B}_x$ and $\mathcal{B}_y$ are neglected because the terrain counter-acts its effects.

The traction forces produced by wheel torque actuation may also output a moment around COM with respect to $\mathcal{B}$.

$$\mathbf{m}_t = \sum_{i=1}^4 \mathbf{c}_i \times (\mathbf{R}_{\mathcal{T}_i}^{\mathcal{B}} \mathbf{t}_i) \quad (3.18)$$

, where $\mathbf{c}_i$ is a vector connecting the mobile robot COM to the contact point c_i , with respect to $\mathcal{B}$. Due to assumption 5, $\mathbf{m}_t$ does not include τ_i .

Finally, if the sum of moments and forces is presented with respect to the actuation τ_i , it comes that

$$\mathbf{f}_t = \sum_{i=1}^4 \mathbf{R}_{\mathcal{T}_i}^{\mathcal{B}} (\mathbf{S}^T(\mathbf{w}_i) \mathbf{R}_{\mathcal{T}_i}^{\mathcal{B}})^{\dagger} \tau_i \quad (3.19)$$

$$\mathbf{m}_t = \sum_{i=1}^4 \mathbf{S}(\mathbf{c}_i) \mathbf{R}_{\mathcal{T}_i}^{\mathcal{B}} (\mathbf{S}^T(\mathbf{w}_i) \mathbf{R}_{\mathcal{T}_i}^{\mathcal{B}})^{\dagger} \tau_i \quad (3.20)$$

3.4.2 Wheel spin rate

Each wheel is considered to spin only if a torque τ_i is fed to the wheel along its lateral axis.

$$\mathbf{I}_{\text{wheel}} \dot{\omega}_i = \tau_i \quad (3.21)$$

Equation 3.21 is expressed with respect to $\mathcal{B}$.

Assumption 6. *The wheel is considered to be locked, so its motion is only due to actuation control.*

3.4.3 Wheel-terrain contact model

The ability to accurately model the interaction forces and moments between a wheel and the contact terrain is of the utmost importance, in order to simulate a theoretical model of a robot or to control a real-life application at a given environment.

There are four possible cases of wheel-terrain interaction (figure 3.5): rigid wheel on rigid terrain, rigid wheel on deformable terrain, deformable wheel on rigid terrain and a deformable wheel on deformable terrain. In [1, 18], one can find analytical models for all these possibilities.

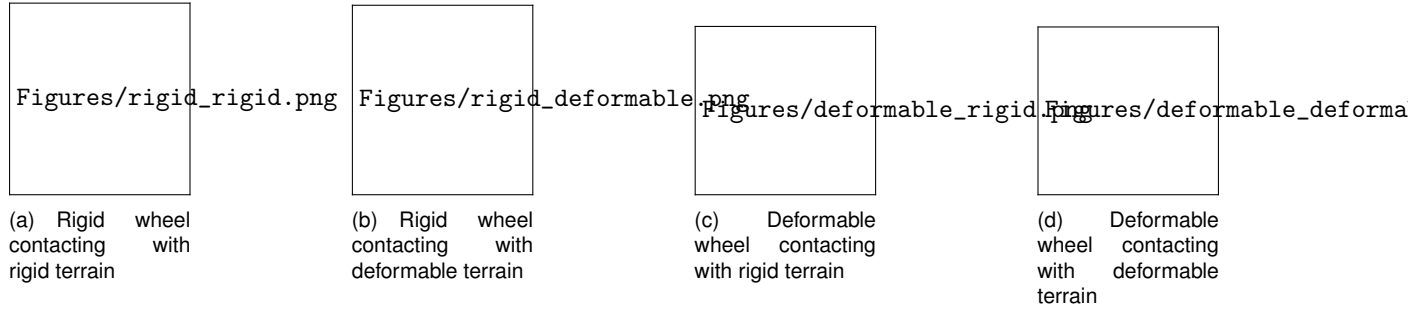


Figure 3.5: Wheel-terrain contact models [1]

In this thesis, only rigid wheels are considered. Even if a wheel presents some degree of deformability (i.e. pneumatic tire), it can be modeled as a rigid wheel if the average tire ground pressure is higher than the critical ground pressure, p_{gr} [1].

When it comes to the type of soil, a rover may traverse deformable (sand) or rigid terrain (rock) when it is deployed at a realistic planetary environment, so it is difficult to predict *a priori* what kind of soil it will be traversing as well as its properties.

The theory of terramechanics focuses on studying the mechanical properties of soils and to assess its behaviour, when it is subject to the influence of a robot with a locomotion set. Apart from it, terramechanics models also deliver performance quantification by describing the stress fields that the contact modules are exposed to. In case of deformable soils, the standard model descriptor is the Bekker-Wong model. On the other hand, the Coulomb friction model fits well to describe the contact between rigid wheels and rigid terrain.

In this thesis, deformable terrain contact physics is discarded as a simulation option because the physics engine of Gazebo, ODE, does not provide a test-bed for it, but also due to the fact that it is

cumbersome to online estimate the parameters of deformable terrain. There are open-source terramechanics real-time simulators (Project Chrono, Vortex) which could enable the simulation of deformable terrain, however these simulators do not provide a bridge to ROS, which is used as a message transmission mechanism between nodes to exchange data among multiple parallel processes. The Coulomb friction model is implemented within ODE to match as close as possible the contact physics between rigid objects.

Coulomb friction model

This model provides a friction force, $\mathbf{f}_f$, acting at each wheel, as a result of the action of the soil on the wheel. This forces acts at c_i with a direction opposite to the wheel velocity $\mathbf{v}_i$. $\mathbf{f}_f$ breaks into a longitudinal ($f_{f,x}$) and lateral ($f_{f,y}$) component, in case f_f is expressed with respect to $\mathcal{T}_i$. Apart from this, a terrain reaction force $\mathbf{f}_n$, acts at c_i , with a direction parallel to the contact normal at c_i , so $\mathbf{f}_n$ only holds a single vertical component within $\mathcal{T}^i$. The Coulomb friction model states the following,

$$\begin{cases} \left(\frac{f_{f,x}}{\mu_x f_n} \right)_i^2 + \left(\frac{f_{f,y}}{\mu_y f_n} \right)_i^2 \leq 1 \\ (f_n)_i > 0 \end{cases}, \quad i = 1, \dots, 4 \quad (3.22)$$

Equation 3.22 states that $(\mathbf{f}_f)_i$ lies within an ellipse with diagonals $\mu_x (f_n)_i$ and $\mu_y (f_n)_i$. In case $\mu_x = \mu_y$, the ellipse turns into a circle. μ_x and μ_y denote friction coefficients along the longitudinal and lateral components of $\mathcal{T}^i$, respectively.

Assumption 7. $\mu_x = \mu_y$.

From assumption 7, and if one states $(f_f)_i^2 = (f_{f,x})_i^2 + (f_{f,y})_i^2$, it comes that,

$$(f_f)_i \leq \mu (f_n)_i \quad (3.23)$$

If a wheel i sticks to the contact point c_i , it means equation 3.23 is inactive. On the other hand, if a wheel is sliding, equation 3.23 is active and $(\mathbf{f}_f)_i$ has an opposite direction to $(\mathbf{v}_i)_{\mathcal{T}^i}$.

$$\begin{cases} (\mathbf{f}_f)_i = -(\mathbf{t}_i + \mathbf{g}_i), \quad \mathbf{v}_i = \mathbf{0} \wedge \mathbf{a}_i = \mathbf{0} \\ (\mathbf{f}_f)_i = -\mu (f_n)_i \cdot \left(\frac{\mathbf{v}_i}{\|\mathbf{v}_i\|} \right)_{\mathcal{T}^i}, \quad \mathbf{v}_i \neq \mathbf{0} \\ (\mathbf{f}_f)_i = -\mu (f_n)_i \cdot \left(\frac{\mathbf{a}_i}{\|\mathbf{a}_i\|} \right)_{\mathcal{T}^i}, \quad \mathbf{v}_i = \mathbf{0} \wedge \mathbf{a}_i \neq \mathbf{0} \end{cases} \quad (3.24)$$

In order to introduce equation 3.24 into an ODE, its discontinuities need to be smoothed. From this observation, $\mathbf{v}_i$ is kept as a *breaker*, and the role of $\mathbf{a}_i$ in the Coulomb friction model is discarded for the sake of simplicity, thus eliminating the last line of equation 3.24. Motivated from this statement, equation 3.24 is transformed into,

$$(\mathbf{f}_f)_i = -(\mathbf{f}_t + \mathbf{f}_g + \mathbf{f}_n)_i \cdot (1 - \sigma(\|\mathbf{v}_i\|)) - \mu (f_n)_i \cdot \frac{\mathbf{v}_i}{\|\mathbf{v}_i\| + \epsilon} \cdot \sigma(\|\mathbf{v}_i\|) \quad (3.25)$$

, where $\sigma(\cdot)$ denotes the sigmoid function, with $k = 100$, and $\epsilon = 1e^{-6}$, denotes a very small value that keeps $\frac{\mathbf{v}_i}{\|\mathbf{v}_i\|}$ differentiable at $\|\mathbf{v}_i\| = 0$.

Then, the sum of friction forces $\mathbf{f}_f$ acting on the mobile robot is given by,

$$\mathbf{f}_f = \sum_{i=1}^4 \mathbf{R}_{\mathcal{T}^i}^{\mathcal{B}} (\mathbf{f}_f)_i \quad (3.26)$$

Finally, the determination of $(\mathbf{f}_n)_i$ is made by computing the portion of the mobile robot weight every wheel i supports, which is presented below.

Effects of gravity

Gravity pushes the mobile robot into the terrain. As a response, ground reaction forces $(\mathbf{f}_n)_i$ sustain the mobile robot. At each wheel i , the effect of gravity acting at c_i is denoted by $(\mathbf{f}_g)_i$, which is expressed with respect to $\mathcal{T}_i$. The vertical component of $(\mathbf{f}_g)_i$ equals $(\mathbf{f}_n)_i$ and are orthogonal to $(\mathbf{f}_f)_i$ and $\mathbf{t}_i$. The sum of $(\mathbf{f}_g)_i$ for every wheel equals the mobile robot weight .

$$m\mathbf{g} = \mathbf{R}_{\mathcal{B}}^{\mathcal{W}} \sum_{i=1}^4 \mathbf{R}_{\mathcal{T}^i}^{\mathcal{B}} (\mathbf{f}_g)_i \quad , \quad (f_{g,z})_i = -(f_{n,z})_i \quad (3.27)$$

, where $\mathbf{g}$ denotes the gravity vector with respect to $\mathcal{W}$. The sum of the effects of gravity and ground reaction forces with respect to $\mathcal{B}$ are stated by,

$$\mathbf{f}_g = \sum_{i=1}^4 \mathbf{R}_{\mathcal{T}^i}^{\mathcal{B}} (\mathbf{f}_g)_i \quad \mathbf{f}_n = \sum_{i=1}^4 \mathbf{R}_{\mathcal{T}^i}^{\mathcal{B}} (\mathbf{f}_n)_i \quad (3.28)$$

3.5 Full state-space model

In order to sum up the previous sections, it is possible to deliver a full state-space model with respect to the state vector $\mathbf{x} = [\mathbf{p}^T \quad \Psi^T \quad \mathbf{v}^T \quad \omega^T]^T$.

$$\mathbf{f}(t, \mathbf{x}(t), \mathbf{u}(t)) = \begin{cases} \dot{\mathbf{p}} = \mathbf{R}_{\mathcal{B}}^{\mathcal{W}} \mathbf{v} \\ \dot{\Psi} = \bar{\mathbf{R}}_{\mathcal{B}}^{\mathcal{W}} \omega \\ \dot{\mathbf{v}} = -\omega \times \mathbf{v} + \frac{1}{m} \mathbf{f} \\ \dot{\omega} = \mathbf{I}^{-1} (-\omega \times (\mathbf{I}\omega) + \mathbf{m}) \end{cases} \quad (3.29)$$

Motivated from equations 3.19, 3.20, 3.26 and 3.28, $\mathbf{f}$ and $\mathbf{m}$ are represented with respect to the actuation wheel torques, τ_i .

$$\mathbf{m}_{(\mathbf{f}_n)} = \mathbf{m}_{(\mathbf{f}_g)} = \mathbf{m}_{(\mathbf{f}_f)} = \mathbf{0}$$

$$\begin{bmatrix} \mathbf{f} \\ \mathbf{m} \end{bmatrix} = \begin{bmatrix} \mathbf{A}_1 & \mathbf{A}_2 & \mathbf{A}_3 & \mathbf{A}_4 \\ \mathbf{S}(\mathbf{c}_1) \mathbf{A}_1 & \mathbf{S}(\mathbf{c}_2) \mathbf{A}_2 & \mathbf{S}(\mathbf{c}_3) \mathbf{A}_3 & \mathbf{S}(\mathbf{c}_4) \mathbf{A}_4 \end{bmatrix} \begin{bmatrix} \tau_1 \\ \tau_2 \\ \tau_3 \\ \tau_4 \end{bmatrix} + \mathbf{1}_{6 \times 6} \begin{bmatrix} \mathbf{f}_f & \mathbf{f}_g & \mathbf{f}_n \\ \mathbf{0} & \mathbf{0} & \mathbf{0} \end{bmatrix} \mathbf{1}_{3 \times 3} \quad (3.30)$$

, where $\mathbf{A}_i = \mathbf{R}_{\mathcal{T}_i}^B (\mathbf{S}^T(\mathbf{w}_i) \mathbf{R}_{\mathcal{T}_i}^B)^\dagger$.

3.6 Model verification

ni

3.6.1 Maximum inclination

Chapter 4

Background

4.1 Two-layer map generation

Elevation data is stored as a **Digital Elevation Models**, which is defined by a compact, discrete function $e_d : \mathcal{E}'_x \times \mathcal{E}'_y \rightarrow \mathcal{E}'_z$. Similarly, a **discrete traversability map** $\mathcal{T}'$ is defined by a function $t_d : \mathcal{T}'_x \times \mathcal{T}'_y \rightarrow \mathcal{T}'_z$, where $\mathcal{E}'_k$ and $\mathcal{T}'_k$ define each discrete map domain on the respective Degree of freedom. As mentioned in section 3.2.1,

$$\mathcal{E}'_x \times \mathcal{E}'_y = \mathcal{T}'_x \times \mathcal{T}'_y \quad (4.1)$$

, so t_d and e_d can be encapsulated into a **two-layer discrete map function** $m'_2 : \mathcal{M}'_{2,x} \times \mathcal{M}'_{2,y} \rightarrow \mathcal{E}'_z \times \mathcal{T}'_z$ (equation 4.2), where $\mathcal{M}'_{2,x} \times \mathcal{M}'_{2,y}$ swaps the equality introduced in 4.1 by an unique formulation.

$$(e_d(x_i, y_j), t_d(x_i, y_j)) = m_2(x_i, y_j) = (h_d, c_d), \quad i = 0, \dots, n_x - 1 \quad j = 0, \dots, n_y - 1 \quad (4.2)$$

, where n_x and n_y stand as the number of divisions in the longitudinal and lateral directions of maps $\mathcal{E}'$ and $\mathcal{T}'$ along frame $\mathcal{W}$, respectively. $x_i \in \mathcal{M}'_{2,x}$ and $y_i \in \mathcal{M}'_{2,y}$, denote the longitudinal and lateral positions of the discrete two-layer map $\mathcal{M}'_2$ with respect to frame $\mathcal{W}$. $\mathcal{E}_k, \mathcal{T}_k$ denote continuous subsets of $\mathbb{R}$, which encapsulate each map limit on the respective Degree of freedom. Then, if functions e_d and t_d are interpolated, a **continuous elevation map function** $e : \mathcal{E}_x \times \mathcal{E}_y \rightarrow \mathcal{M}_z$ and a **continuous traversability map function** $t : \mathcal{T}_x \times \mathcal{T}_y \rightarrow \mathcal{T}_z$ can be encapsulated into a **two-layer continuous map function** $m_2 : \mathcal{M}_{2,x} \times \mathcal{M}_{2,y} \rightarrow \mathcal{E}_z \times \mathcal{T}_z$,

$$(e(x, y), t(x, y)) = m_2(x, y) = (h, c), \quad x \in \mathcal{M}_{2,x}, \quad y \in \mathcal{M}_{2,y} \quad (4.3)$$

A discrete or continuous **traversability map** can be obtained from e_d or e through an algorithm $\mathbb{T}$, which is presented in section [insert section; draw picture of 2-layer map](#).

4.2 Fast Marching Method

In this thesis, a fast marching method is employed in order to serve as the framework to extract optimal reference paths. This method relies on solving a boundary-value problem [19], which yields a potential field with a wave that starts at a pre-defined source and expands throughout a map $\mathcal{M}_1 : \mathbb{R}^2 \mapsto \mathbb{R}$, which is a reduction of a layered map $\mathcal{M}_n : \mathbb{R}^2 \mapsto \mathbb{R}^n$.

Mathematically, this comes to solving the Eikonal equation,

$$\begin{aligned} |\nabla u(x, y)| f(x, y) &= 1 \\ f(\Gamma) &= 0 \end{aligned}, \quad (x, y), \Gamma \in D_{\mathcal{M}_1} \quad (4.4)$$

, where $u(\cdot)$ is the solution to the Eikonal equation, (x, y) is a position in the map, $f(\cdot)$ is a speed (or cost) function [10] which contains information regarding the set of points where the wave delays or rushes its progression throughout $\mathcal{M}_2$. $\Gamma \subset \mathcal{M}_2$ is the wave front at the source. Due to the fact that $u(x)$ delivers the time a wave needs to reach x from the source $x_s \in \mathcal{M}_2$, it is trivial that $u(\Gamma)$ equals zero because Γ is small ball around x_s . On the other hand, $u(x) = T$ provides the set of points from where it takes a time T to reach x_s .

Then, it is possible to find an optimal time path from x to x_s which deviates from the points where F has the lowest values. The direction of a path at some point is perpendicular to the wave front at that point, which is provided by the gradient $-\Delta u(x)$. The function F is computed offline according to the application, and then it is inserted into 4.4. After that, the eikonal problem is solved (also offline) and delivers a potential field with a source x_s . From there, it is possible to compute online the fastest and safest path to x_s at the position the mobile robot is in. More information on this issue is provided at

Chapter 5

Global path planner

In order to feed the control system with a reference path, an offline survey of the target environment is conducted in order to generate a **traversability cost map**, which performs an analysis of the terrain unevenness with respect to the robot kinematics and geometric features. Then, the same **cost map** is fed to the Eikonal equation in order to yield a gradient potential map, where an optimal path to some goal position is retrieved.

5.1 Traversability map computation

As mentioned in subsection 2.1.1, there is no explicit guideline on how to generate a traversability map. In this thesis, several approaches are compared. First, the state-of-the art algorithm from [7] is reviewed and evaluated. Then, this algorithm is reformulated to account for other metrics such as terrain inclination and roughness [9].

5.1.1 Height minimization

A constrained optimization problem which minimizes the height of the robot at each cell is described. It is very similar to the solution provided in [7]. In this case, a wheeled vehicle is used instead of a tracked one. The yaw angle is set as a fixed variable and the initial guess v_0 of the problem at every cell is set by an approximation to the optimal solution.

Initial guess

Provided that the yaw angle ψ and the x and y position coordinates are known, each wheel point is projected to $\mathcal{M}_2$. Then, from the group of retrieved contact points, the lowest height one is discarded, and an unit, normal vector to a plane which contains these points, is computed. Finally, the roll and pitch of this plane are given by equation 5.1, where (a, b, c) are the coordinates of a normal vector to the above mentioned plane. In case any of these angles are higher than ϕ_{max} or θ_{max} , the respective angle is set to the highest admissible value, so that the initial guess is feasible. The height initial guess

z_0 is set as the sum of the projection of the robot center of mass point to $\mathcal{M}_2$ plus the robot geometric height, to avoid any possible intrusion of the vehicle hull into the ground.

$$\begin{aligned}\phi &= \arcsin(\sin(\psi) a - \cos(\psi) b) \\ \theta &= \arccos\left(\frac{c}{\cos(\phi)}\right)\end{aligned}\tag{5.1}$$

Non-linear programming

The minimization of the vehicle Center of mass is expressed as a non-linear optimization problem (NLP) in equation 5.2, where Ψ_{max} stands as the limits for each euler angle. $\mathbf{p}_k$ stands as the k hull point position, which is defined with respect to $\mathcal{W}$, by taking into account the robot geometry. $[x, y, \psi]^T$ are fixed variables whereas $[z, \phi, \theta]^T$ are set as optimization variables. Each point included in 5.2 adds an additional constraint by stating each hull point must be above or at contact with the terrain. In case less than three hull points (each one from a different wheel) are at contact with the terrain, the terrain inclination returned by algorithm 1 is $\frac{\pi}{2}$. This expresses the fact that the robot will stand at an unstable position, so this position is regarded as an undesirable traversable position. On the other hand, the terrain inclination is given by equation 5.3, which returns the angle between a normalized vector with direction B_z and the normalized vector $\mathcal{W}_z$ [7]. Although algorithm 1 can be extended to any kind of wheeled-mobile robot configuration, only four wheel mobile robots are considered in this thesis, as mentioned in section 3.1.

$$\begin{aligned}\text{Minimize} \quad & J = z \\ \text{w.r.t.} \quad & \mathbf{v} = (z, \phi, \theta) \\ \text{subject to} \quad & \forall_k : z_k - e(x_k, y_k) \geq 0 \quad (5.2) \\ & \Psi_{max} - |\Psi| \geq 0 \\ & c = \arccos(\cos(\theta) \cos(\phi)) \quad (5.3)\end{aligned}$$

Algorithm 1 Terrain inclination with HEIGHTMIN

```

1: function TERRAINSLOPE( $\Psi_{max}, e(\cdot), \mathbf{v}_0, x, y, \psi$ )
2:    $\mathbf{v}_* \leftarrow \text{HEIGHTMIN}(\mathbf{v}_0, \Psi_{max}, e(\cdot), x, y, \psi)$   $\triangleright$  solve 5.2
3:   if Contacts  $\geq 3$  then
4:      $c \leftarrow \text{INCLINATION}(\phi_*, \theta_*)$   $\triangleright$  5.3
5:   else
6:      $c \leftarrow \frac{\pi}{2}$ 
7:   end if
8:   return  $c, \mathbf{v}_*$ 
9: end function
```

Algorithm 2 Terrain inclination map with TERRAINSLOPE

```

1: function INCLINATIONFIXYAW( $\Psi_{max}, \psi, e(\cdot), n_x, n_y$ )
2:   for each  $\{i, j\} \in \{0, \dots, n_x - 1\} \times \{0, \dots, n_y - 1\}$  do
3:      $\mathbf{v}_0 \leftarrow \text{INITIALGUESS}(x_i, y_j, \psi)$   $\triangleright$  feasible initialization
4:     if  $\mathcal{C}_2 \subset \mathcal{M}_2$  then
5:        $c, \mathbf{v}_* \leftarrow \text{TERRAINSLOPE}(\Psi_{max}, e(\cdot), \mathbf{v}_0, x_i, y_j, \psi)$   $\triangleright$  algorithm 1
6:     else
7:        $c \leftarrow \frac{\pi}{2}$ 
8:     end if
9:      $t_d(i, j) \leftarrow c$ 
10:  end for
11:  return  $t_d(\cdot)$ 
12: end function
```

Then, algorithm 2 extends the application of algorithm 1 to each cell of the elevation grid. It should be noted that the discrete elevation grid $e_d(\cdot)$ shall have such dimensions that the cell division length is smaller than the magnitude order of the wheel diameter in meters. Moreover, a convex area $\mathcal{C}_2$ (equation 5.4) centered at $[x, y]^T$ which captures the robot hull $\mathcal{H}_2$ projection in $\mathcal{M}_2$ must lie inside the map domain. If this condition is not met, the terrain inclination angle assigned to $t_d(\cdot)$ is $\frac{\pi}{2}$. Therefore, positions at which the assigned terrain inclination is $\frac{\pi}{2}$ are considered to be untraversable. To sum up, the higher the inclination, the less desirable it is to pass through the respective position.

$$\mathcal{C}_2 = \{\min \underline{h} : \|\mathbf{p}_2 - \mathbf{a}\| \leq \underline{h} \mid \mathbf{p}_2, \mathbf{a} \in \mathcal{M}_2, \mathbf{p}_{2,k} \in \mathcal{C}_2\} \quad (5.4)$$

Variable yaw

The terrain traversability may also be assessed by finding the maximum inclination at each position by saving the maximum retrieved inclination among several yaws. Algorithm 2 lacks a thorough evaluation of the terrain irregularity, as it changes depending on the mobile robot heading. In order to tackle this absence of completeness, algorithm 2 is reformulated to address this issue.

Algorithm 3 Terrain inclination map with TERRAIN SLOPE and variable yaw

```

1: function INCLINATIONVARYAW( $\Psi_{max}, e(\cdot), n_x, n_y$ )
2:   for each  $\{i, j\} \in \{0, \dots, n_x - 1\} \times \{0, \dots, n_y - 1\}$  do
3:     for each  $\psi_k$  do ▷ variable yaw
4:        $\mathbf{v}_0 \leftarrow \text{INITIALGUESS}(x_i, y_j, \psi_k)$  ▷ feasible initialization
5:       if  $\mathcal{C}_2 \subset \mathcal{M}_2$  then
6:          $c_k, (\mathbf{v}^*)_k \leftarrow \text{TERRAIN SLOPE}(\Psi_{max}, e(\cdot), \mathbf{v}_0, x_i, y_j, \psi_k)$  ▷ algorithm 1
7:       else
8:          $c_k \leftarrow \frac{\pi}{2}$ 
9:       end if
10:    end for
11:     $c \leftarrow \max(c_k)$  ▷ save maximum inclination
12:     $t_d(i, j) \leftarrow c$ 
13:  end for
14:  return  $t_d(\cdot)$ 
15: end function

```

In algorithm 3, $\psi_k = \{-\pi, \dots, -\pi + k\alpha, \dots, \pi - \alpha\}$, where α stands as the step angle of the set ψ_k .

5.1.2 Maximum terrain roughness

Terrain traversability may also be evaluated by inspecting terrain roughness around each position of the map, where the goal is to evaluate the maximum terrain unevenness at each position of the map. In this thesis, terrain roughness at each position of $\mathcal{M}_2$ is defined as the standard deviation of elevations inside $\mathcal{H}_2$.

$$\bar{e}_d(i, j) = \frac{\sum_{x_i, y_j \in \mathcal{H}_2} e(x_i, y_j)}{n(\mathcal{H}_2)}, \mathcal{H}_2 = \{(x_i, y_j) \in \mathcal{H}_2\} \quad (5.5)$$

, where $\mathcal{H}_2$ denotes a finite set of points from $e_d(\cdot)$ inside $\mathcal{H}_2$ and $\bar{e}_d(i, j)$ the average of elevations for points inside $\mathcal{H}_2$. Then, terrain roughness is defined as,

$$t_d(i, j) = \sqrt{\frac{\sum_{x_i, y_j \in \mathcal{H}_2} (e(i, j) - \bar{e}_d(i, j))^2}{n(\mathcal{H}_2)}} \quad (5.6)$$

The set of points inside $\mathcal{H}_2$ depends on the yaw angle of the mobile robot, so $t_d(i, j)$ is computed for each yaw angle. Similarly to what was done in algorithm 3, the terrain roughness assigned to the respective cell is the maximum roughness among the set of yaws ψ_k . Finally, a maximum terrain roughness may be computed by coding algorithm 4.

Algorithm 4 Maximum terrain roughness map

```

1: function ROUGHNESS( $\Psi_{max}, e(\cdot), n_x, n_y$ )
2:   for each  $\{i, j\} \in \{0, \dots, n_x - 1\} \times \{0, \dots, n_y - 1\}$  do
3:     if  $\mathcal{C}_2 \subset \mathcal{M}_2$  then
4:       for each  $\psi_k$  do ▷ variable yaw
5:          $c_k \leftarrow$  equation (5.6) ▷ equation (5.6)
6:       end for
7:     else
8:        $c_k \leftarrow \text{None}$ 
9:     end if
10:     $c \leftarrow \max(c_k)$  ▷ save maximum roughness
11:     $t_d(i, j) \leftarrow c$ 
12:  end for
13:  Replace cells assigned with None by  $\max(t_d(\cdot))$ 
14:  return  $t_d(\cdot)$ 
15: end function

```

In case $\mathcal{C}_2 \not\subset \mathcal{M}_2$, the respective cell is assigned with **None**. At the end, these assignments are replaced by the maximum roughness among all cells, in order to enhance the fact that these cells are meant to be among the most untraversable ones.

5.1.3 Worst-case scenario

A more conservative traversability map can be created by assigning to every cell, the highest value among the traversability maps computed by algorithms 3 and 4 at the respective cell. This approach provides a joint awareness of terrain inclination and roughness on a single traversability map. In order to accomplish that, the traversability maps that come into play are normalized in order to enable a fair comparison among maps.

Traversability map refinement

In order to classify high cost borderline areas as undesirable, a refinement of the worst-case scenario map is made, so that a mobile robot avoids a close traverse to undesirable positions. Algorithm 5 presents a way to code this refinement method, by increasing the cost of areas around positions with a cost higher than a defined threshold, C . At each point with cost higher than C , a decreasing, isometrical

source radiates up to a distance r_s from the respective point. At each point which is inside the source, the existing cost is replaced by the source cost $c_s(\cdot)$ in case this one is higher.

$$c_s(x) = -\left(\frac{x}{r_s}\right)^2 + 1, \quad 0 \leq x \leq r_s \quad (5.7)$$

Algorithm 5 Refinement of worst-case map

Require: $\Psi_{max}, e(\cdot), n_x, n_y, r_s, C$

```

1:  $t_{d, inc}(\cdot) \leftarrow \text{INCLINATIONVARYAW}(\Psi_{max}, e(\cdot), n_x, n_y)$  ▷ run algorithm 3
2:  $t_{d, r}(\cdot) \leftarrow \text{ROUGHNESS}(\Psi_{max}, e(\cdot), n_x, n_y)$  ▷ run algorithm 4
3:  $t_{d, inc}(\cdot), t_{d, r}(\cdot) \leftarrow \frac{t_{d, inc}(\cdot)}{\max(t_{d, inc}(\cdot))}, \frac{t_{d, r}(\cdot)}{\max(t_{d, r}(\cdot))}$  ▷ Normalize maps between 0 and 1
4: for each  $\{i, j\} \in \{0, \dots, n_x - 1\} \times \{0, \dots, n_y - 1\}$  do
5:    $t_d(i, j) \leftarrow \max(t_{d, inc}(i, j), t_{d, r}(i, j))$  ▷ generate worst-case map
6: end for
7: for each  $\{i, j\} \in \{0, \dots, n_x - 1\} \times \{0, \dots, n_y - 1\}$  do
8:   if  $t_d(i, j) \geq C$  then
9:     Add  $(i, j)$  to  $p$  ▷ Add point with cost above or equal to  $C$  to list
10:  end if
11: end for
12: for each  $\{i, j\} \in p$  do
13:   for each  $\{x_{i_2}, y_{j_2}\} \in \{x_i - r_s, \dots, x_i + r_s\} \times \{y_j - r_s, \dots, y_j + r_s\}$  do
14:    if  $\{x_{i_2}, y_{j_2}\} \in \mathcal{M}_2$  then
15:       $d \leftarrow \sqrt{(x_{i_2} - x_i)^2 + (y_{j_2} - y_j)^2}$  ▷ compute distance between  $(i, j)$  and  $(i_2, j_2)$ 
16:      Add  $c_s(d)$  to  $T_d(i_2, j_2)$  ▷ add equation 5.7 result to list of source costs which concern  $(i_2, j_2)$ 
17:    end if
18:  end for
19: end for
20: for each  $\{i, j\} \in \{0, \dots, n_x - 1\} \times \{0, \dots, n_y - 1\}$  do
21:    $t_d(i, j) \leftarrow \max(T_d(i, j))$  ▷ Assign highest cost among source costs and existing cost
22: end for

```

5.1.4 Implementation

In this subsection, the implementation of the algorithms presented above is described, alongside the presentation of results. The elevation maps used in this implementation are the target environments, which will serve as a simulation test-bed for the control system, later on. These maps contain regions with a variety of terrain, that range from irregular to smooth areas, but also from hilly to planar locations. In figure B.1, these elevation maps are rendered. The wheeled mobile robot Pioneer 3AT was chosen as the test vehicle. The mobile robot dimensions are presented in table 5.1, whereas the elevation maps features are presented in table 5.2.

Table 5.1: Pioneer 3-AT dimensions.

r [m]	l_w [m]	b_w [m]
0.111	0.268	0.394

Table 5.2: Elevation maps dimensions.

Map 1					Map 2				
h_{max} [m]	l_x [m]	l_y [m]	n_x	n_y	h_{max} [m]	l_x [m]	l_y [m]	n_x	n_y
6.58	47.0	47.0	500	500	0.36	0.71	3.18	500	500

The origin of each elevation map is considered to be an inertial world frame $\mathcal{W}$, which is placed at the center of each elevation map, as it is depicted in figure B.1. Typically, the original files of elevation maps come as .tif format or .png with grayscale encoding. Then, a suitable rescaling shall be made in order to yield a large enough map, with a meters per pixel discretization such that the wheel diameter is of the same magnitude order. Moreover, the maximum height shall be chosen in such a way that non-negligible crossable slopes are on the map.

Each algorithm is coded in Python, whereas the NLP presented in 5.2 is implemented with the casADi numerical optimization tool [20], where IPOPT [21] is selected as solver. Two different sets of simulations are performed for each map: the first only considers a single contact point at each wheel, whereas the second one takes a set of five hull points at each wheel (figure 5.1).

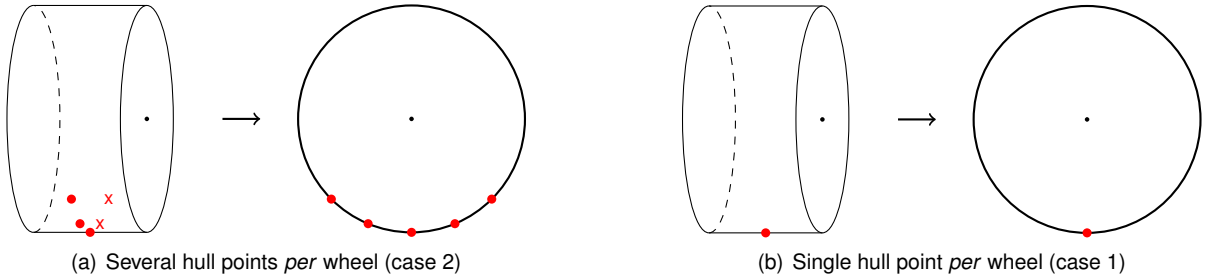


Figure 5.1: Hull points on each wheel

Apart from the restriction that every hull point must be above or at contact with the elevation map surface, the euler angle restrictions are linear and symmetrical. That means the lower and upper bound for each euler angle restriction is symmetrical (table 5.3).

Below, the computed traversability maps are presented for case 1, under the conditions presented in this subsection. Traversability maps that match the results ranging from algorithm 2 to 5 are presented, in order to enhance the differences among all maps. All the results presented below were obtained on an Intel® Xeon® E5-1620 v2 $\times$ 8 processor.

Table 5.3: Euler angles restrictions.

ϕ_{max} [rad]	θ_{max} [rad]	ψ_{max} [rad]
$\frac{\pi}{4}$	$\frac{\pi}{4}$	π

The traversability maps which were obtained for case 1 are rendered in figure 5.2 and 5.3. For the sake of brevity, the maps computed for case 2 are presented in annex B.2, as they are graphically similar to the maps presented in figures 5.2 and 5.3. The maximum terrain roughness map is the same both in case 1 and 2.

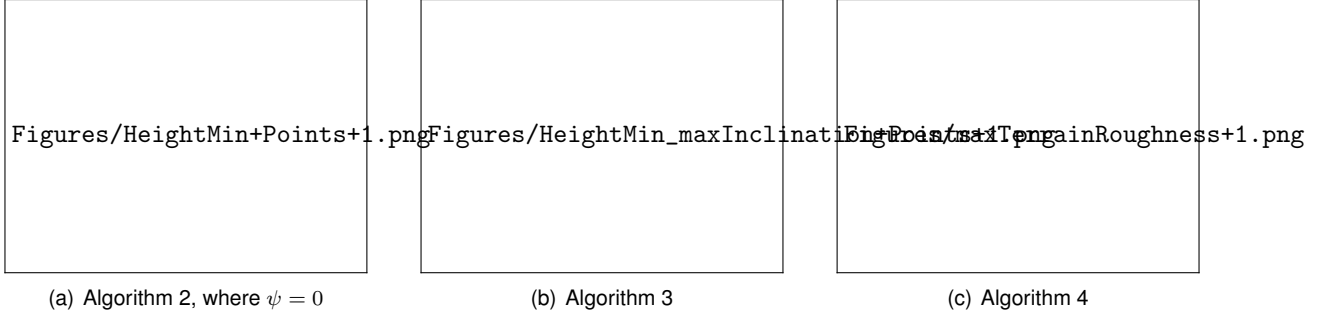


Figure 5.2: Traversability maps (case 1; map 1)

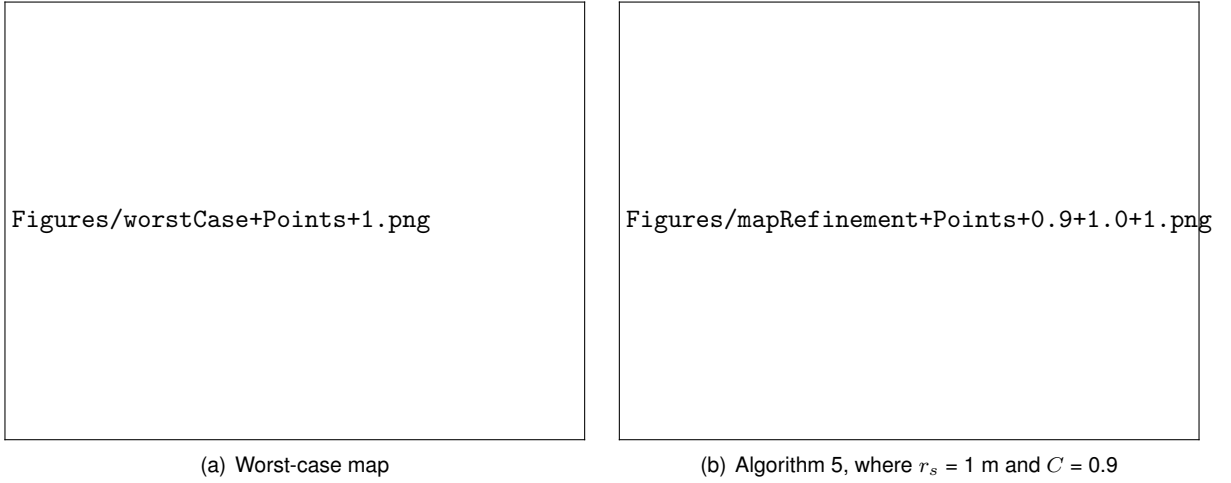


Figure 5.3: Traversability maps (case 1; map 1)

5.1.5 Discussion

Algorithms 2 to 5 were built in such a way that the output map assigns higher costs to hilly or irregular terrain. On the other hand, it was required that the output map captures the terrain height gradient, although it is advisable to assign higher costs to positions which are close to untraversable areas. Then, a balance between a traversability map that leads to the computation of a safe path and the capture of the terrain morphology comes into play.

The analysis of figures 5.2 and 5.3 as well as the results of tables 5.4 and 5.5 express the fact the maps computed with algorithm 2 to 4 capture well the terrain morphology as the correlation between the height gradient and the cost is very high on all cases. However, that correlation decreases a lot when algorithm 5 is applied, due to that the map refinement assigns a higher cost to positions which are closer to untraversable positions. This is price to pay in case one wants the robot to avoid hazardous positions. The correlation $\nabla e(\cdot)$ vs $t(\cdot)$ will depend on the parameters r_s and C . In case r_s is high and C is low, the source that radiates from each point with cost higher or equal to C is larger and applied to a larger set of points, which causes the correlation $\nabla e(\cdot)$ vs $t(\cdot)$ to be smaller.

As it is expected, the computational burden increases a lot, when algorithms 2 and 3 are run for case 2, because the NLP needs to account for more constraints. However, the computation time is not a relevant result because these algorithms are intended to be run offline, prior to any simulation.

Table 5.4: Comparison between algorithms 2 and 3 (Map 1)

Algorithm	Case 1		Case 2	
	2	3	2	3
[ms] / cell	5.82 ± 1.58	71.11 ± 9.96	10.10 ± 3.00	110.33 ± 13.68
iterations / cell	8.99 ± 2.13	118.17 ± 12.36	11.71 ± 2.79	151.36 ± 14.45
optimality [$\times 10^{-3}$]	8.63 ± 43.82	16.31 ± 47.84	5.51 ± 39.37	7.10 ± 40.04
$\nabla e(\cdot)$ vs $t(\cdot)$ [R]	0.89	0.83	0.90	0.93

Table 5.5: Comparison between algorithms 4 and 5 (Map 1)

Algorithm	4	Case 1		Case 2	
		5 [†]	5 ^{**}	5 [†]	5 ^{**}
$\bar{t}$ / cell [ms]	8.51 ± 1.56	$3.12\text{e-}6 \pm 8.10\text{e-}4$	$1.40 *$	$3.12\text{e-}6 \pm 7.63\text{e-}4$	$1.49 *$
$\nabla e(\cdot)$ vs $t(\cdot)$ [R]	0.96	0.78	0.61	0.50	0.13

[†] Worst-case map * $\frac{\text{Total computation time}}{\text{Number of cells}}$ ** $r_s = 1\text{m}$, $C = 0.9$; (mean) $\pm$ (standard deviation), with confidence interval of 68.27 %, when applicable

Optimality was defined as the KKT Lagrangian gradient norm [?] with respect to the optimization states of the NLP 5.2. Then, optimality evaluation is only useful when an algorithm employs a NLP. Generally, optimalities are close to zero, which expresses the fact that the computed costs relate to optimal solutions of 1. On both cases, the optimality decreases from algorithm 2 to 3 and the standard deviation is higher than the mean optimality, due to the fact that the optimality rises above zero as the inclination approaches one. Less optimal solutions were obtained for algorithm 3 because the optimal solution is selected as the maximum inclination, from a set of solutions of 5.2 for every yaw ψ_k , as it is detailed on algorithm 5. As there is a trend that links less optimal solutions to higher inclinations, less optimal solutions are obtained for algorithm 3, because the cost is selected as the highest inclinations from a set of solutions of 5.2 to every yaw ψ_k (see algorithm 3).

Finally, it is worth to note that the traversability map computed by algorithm 3 for case 2 (figure B.2) finds more untraversable positions in comparison to case 1 (figure 5.2), because it is more difficult to accomodate more hull points on the terrain, which makes the distance from the hull points to the ground higher.

5.2 Optimal path planning

Up to now, it is possible to compute a potential flow with respect to any cost map generated based on the algorithms presented in section 5.1. Function $f(\cdot)$ from equation 4.4 is replaced by a speed map. Then, the solution of the Eikonal equation outputs a function $u(\cdot)$ that holds the time it takes for each the goal position from some position. Therefore, $\nabla u(\cdot)$ enables the computation of an optimal path towards the selected goal. $u(\cdot)$ does not possess local minimums, so there's no risk the computation of the optimal path gets stuck.

However, $f(\cdot)$ is not directly replaced by any cost map generated by algorithms 2 to 5, because

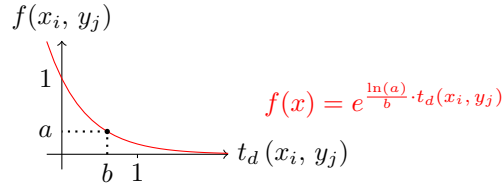


Figure 5.4: Cost to speed conversion (equation 5.8)

lower speed maps must match higher costs. So, these cost maps must be flipped into a speed map beforehand.

$$f(x_i, y_j) = e^{\frac{\ln(a)}{b} \cdot t_d(x_i, y_j)} \quad (5.8)$$

Although this conversion can take many forms, it must comply with the fact that higher costs match lower speeds, so an exponential function of the type e^{-kx} is chosen in this thesis. Equation 5.8 is a particular case of e^{-kx} where $-k = \frac{\ln(a)}{b}$ in order to enforce that if $t_d(x_i, y_j) \geq b$, it comes that $f(x_i, y_j) \leq a$. The choice of a and b is made in such a way, so that the flipping of higher costs from $t_d(x_i, y_j)$ get more penalized on the respective $f(x_i, y_j)$.

Each value that is assigned to every cell on the cost map expresses the fact that it is safer or riskier to pass by that position. If $f(\cdot)$ presents a high (low) value to some cell of the cost map, that means it is safer (riskier) to cross that position. Unlike occupancy maps, speed maps can take any non-negative value at each cell. On both maps, if zero is assigned to some position, that position is *seen* as an obstacle. However, one may enlarge the set of obstacles on $\mathcal{M}_2$ by *masking* (subsection 5.2.1) on $f(\cdot)$ the positions that $t_d(x_i, y_j) \geq c$, where c denotes the maximum admissible cost for traversal.

5.2.1 Implementation

Algorithm 6 sums up the speed map generation steps, whereas algorithm 7 encapsulates the optimal path generation method described in the previous section. Note that N denotes the path length. Algorithm 6 is meant to be run offline, but algorithm 7 is not, so the optimal path generation was coded in C in order to speed up its execution and allow for greater time margins on the iteration window. Then, function `GETOPTIMALPATH( $\cdot$ )` is called in Python using the well-known `ctypes` package.

The computation of a discrete solution $f(\cdot)$ for the eikonal equation is made by applying the method `travel_time()` from the `scikit-fmm` Python package. Finally, the optimal path is obtained by repeatedly finding the minimum time position among the newest path position, until the goal position G is reached or if the path surpasses its maximum size.

As it was noted in the previous section, this method is only valid in case the map discretization is of the same magnitude order of the wheel radius. Table 5.6 presents the parameters which were chosen to generate the results presented in subsection 5.2.2. Maps 1 and 2 are the same as the ones presented in 5.1.

scikit-fmm: <https://github.com/scikit-fmm/scikit-fmm>

Table 5.6: Algorithm 7 parameters.

Map 1			Map 2		
a	b	c	a	b	c
0.2	0.4	0.6	0.36	0.71	3.18

Algorithm 6 Potential flow generation

```

1: function GETPOTENTIALFLOW( $t_d(\cdot)$ ,  $n_x$ ,  $n_y$ ,  $a$ ,  $b$ ,  $c$ )
2:   for each  $\{i, j\} \in \{0, \dots, n_x - 1\} \times \{0, \dots, n_y - 1\}$  do
3:     if  $t_d(x_i, y_j) \geq c$  then
4:       Mask  $f(x_i, y_j)$  ▷ Label this position as an obstacle.
5:     else
6:        $f(x_i, y_j) \leftarrow$  equation 5.8 ▷ Flip cost map to speed map.
7:     end if
8:   end for
9:    $u(\cdot) \leftarrow$  equation 4.4 ▷ Solve eikonal equation
10:  return  $u(\cdot)$ 
11: end function

```

Algorithm 7 Optimal path generation

```

1: function GETOPTIMALPATH( $u(\cdot)$ ,  $S$ ,  $G$ ,  $N$ ,  $\epsilon$ )
2:   if  $S \in \mathcal{O}$  or  $G \in \mathcal{O}$  then
3:     return empty path ▷ Reject path generation in case S or G positions are labeled as obstacles.
4:   end if
5:    $(x_i, y_j) \leftarrow S$ , index  $\leftarrow 0$  ▷ Generate optimal path from starting position S
6:   while  $\|(x_i, y_j) - G\| > \epsilon$  and index  $< N$  do
7:      $(x_i, y_j) \leftarrow$  GETPOSITION ( $\text{MIN}(u(\partial(x_i, y_j)))$ ) ▷ Get minimum time position among contiguous cells.
8:     path.append( $(x_i, y_j)$ ) ▷ Append minimum time position to path.
9:     index  $\leftarrow$  index + 1
10:  end while
11:  while index  $< N$  do
12:    path.append( $(x_i, y_j)$ ) ▷ Fill path with last position until N positions are gathered.
13:    index  $\leftarrow$  index + 1
14:  end while
15:  return path
16: end function

```

5.2.2 Discussion

According to the guidelines described above, a speed map example is generated with respect to a worst-case traversability map for case 1 and elevation map 1 (figure 5.5), by running algorithm 6. Then, a potential flow is generated by computing a solution $u(\cdot)$ to the Eikonal equation 4.4.

In figure 5.5, $-\nabla u(\cdot)$ is rendered all over the map on blue arrows. There, $-\nabla u(\cdot)$ is perpendicular to every isochrone line (black lines), as expected. Instead of applying algorithm 7, optimal paths may also be generated by following $-\nabla u(\cdot)$ until the goal point (red dot) is reached.

Figure 5.6 presents several optimal paths generated with algorithm 7. All these paths meet at the goal point, and follow a gradient descent direction as expected. It is also verified that the computed paths avoid the area defined as obstacle (white). The refinement of the worst-case map does not considerably change the respective potential flow, even if the masked area is greater. This is why the optimal paths presented in figure 5.6 are similar, provided that the starting and goal points are the same. However, this similarity is not a trend. In fact, provided that the parameters r_s and C yield a great change of the masked area and speed maps, the potential flow of a refined map may output a very different optimal path.

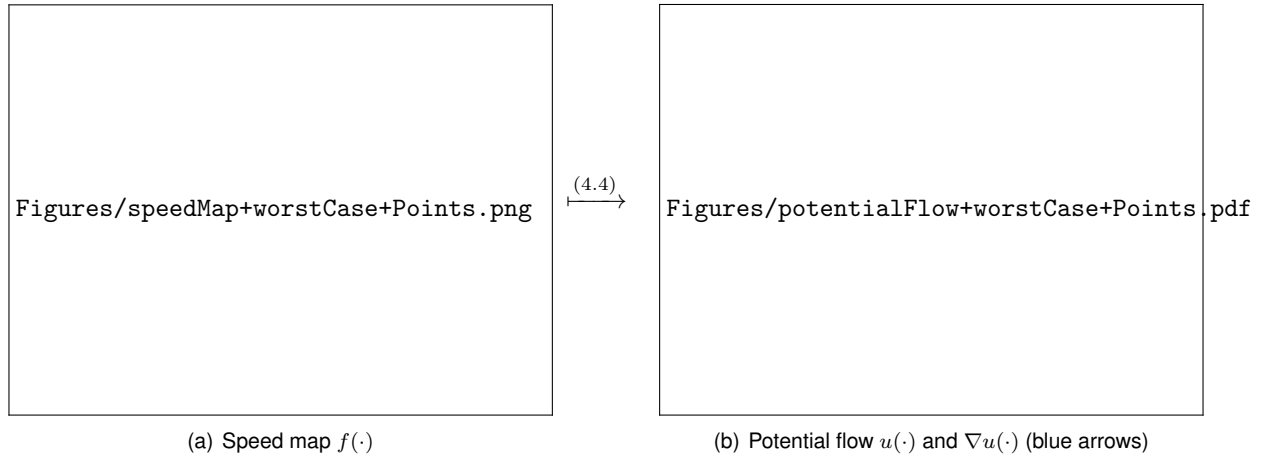


Figure 5.5: Potential flow and speed map for worst-case traversability map (case 1; map 1)

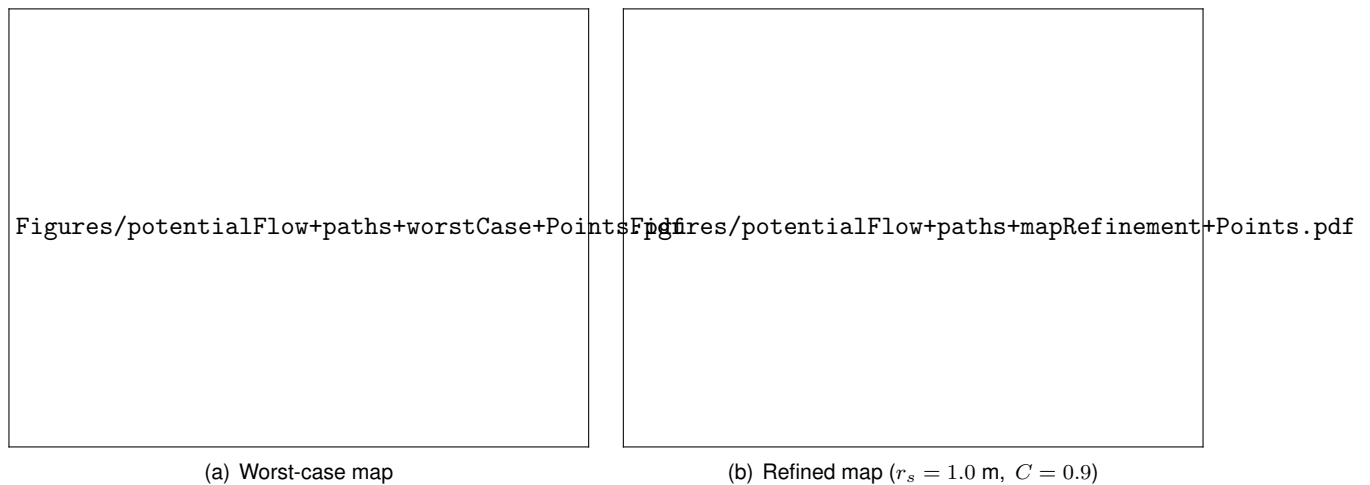


Figure 5.6: Optimal paths (case 1; map 1)

Figure 5.7 underlines once more the fact that, different cost maps yield different speed maps, which in turn yield different potential flows. In the end, different optimal paths are obtained. Due to what was mentioned in the previous paragraph, the optimal paths for the worst-case and respective refinement (case 1) almost match. Although it may happen, this is not a typical case.

The table next to figure 5.7 presents data related to the runtimes of the implementation of function `GETOPTIMALPATH(·)` from algorithm 7 in C. This function was run 27083 times for different maps and

starting positions. The map refinement parameters, equation 5.8 parameters and goal point remained fixed. 800 points are appended for each path. In case $S \in \mathcal{O}$, that runtime was discarded.

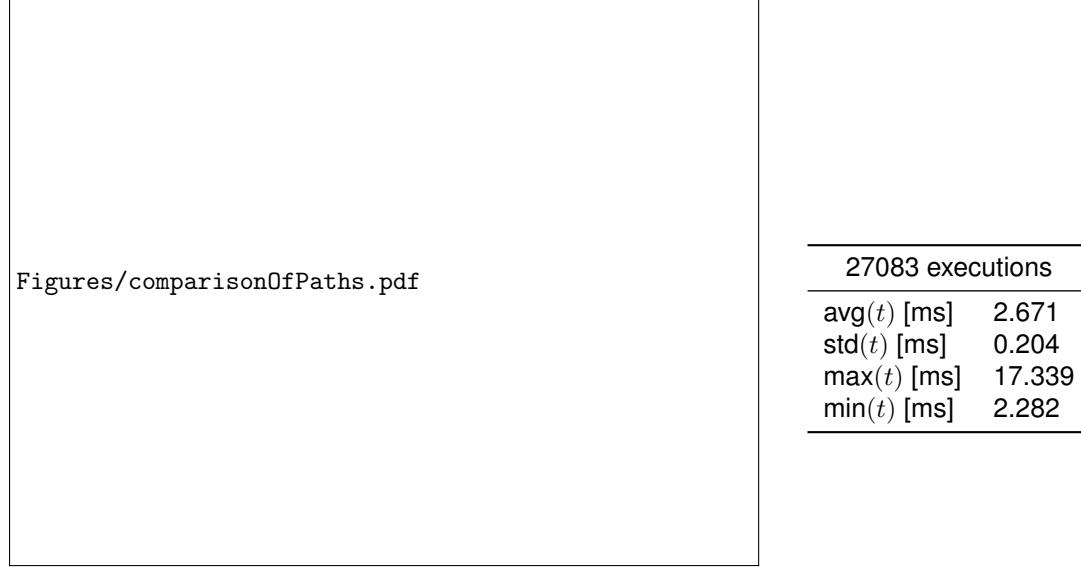


Figure 5.7: Paths comparison and GETOPTIMALPATH($\cdot$) method runtimes.

The results show that function GETOPTIMALPATH($\cdot$) is feasible for implementation within a system control loop, which time window is around 100 ms. In case more than 800 points are appended for the optimal path generation, the runtimes likely increase. Within a real-time guidance system, the starting position S is the current position of the vehicle, whereas the returned path is the reference path to be followed (figure 5.8). All the results presented in this subsection were obtained on an Intel® Xeon® E5-1620 v2 $\times$ 8 processor.

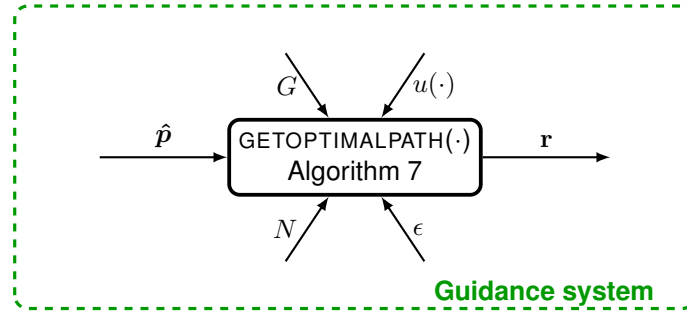


Figure 5.8: Optimal path generation module.

Due to the fact that GETOPTIMALPATH($\cdot$) returns None in case $S \in \mathcal{O}$, one should take care that $\mathcal{O}$ is not scattered along the map, but rather is restricted to *blocks* of obstacles, in order to prevent frequent rejections of an optimal path computation.

Chapter 6

Control system

Insert your chapter material here...

6.1 Problem Description

6.2 Baseline Solution

Analysis of the baseline solution...

6.3 Enhanced Solution

Quest for the optimal solution...

6.3.1 Figures

Insert your section material and possibly a few figures...

Make sure all figures presented are referenced in the text!

The caption should appear below the figure.

6.3.1.1 Images

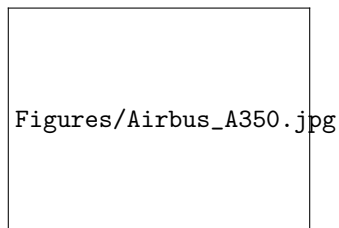
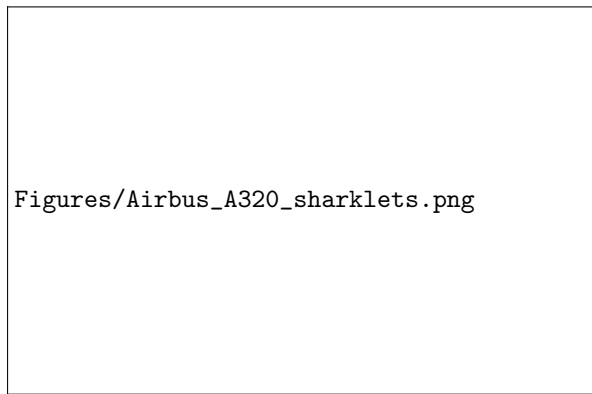


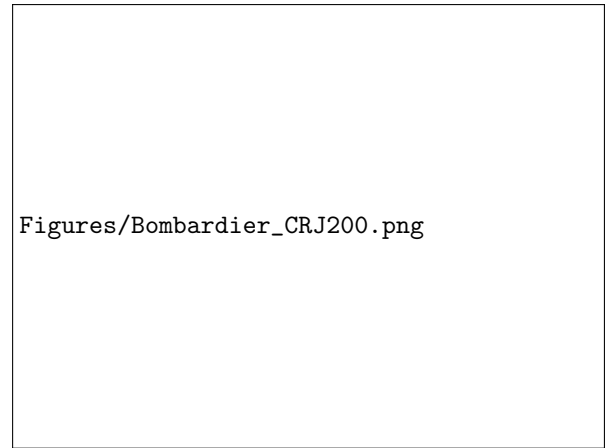
Figure 6.1: Caption for figure.

Make reference to Figures 6.1 and 6.2.

By default, the supported file types are *.png, .pdf, .jpg, .mps, .jpeg, .PNG, .PDF, .JPG, .JPEG*.



(a) Airbus A320



(b) Bombardier CRJ200

Figure 6.2: Some aircrafts.

See http://mactex-wiki.tug.org/wiki/index.php/Graphics_inclusion for adding support to other extensions.

6.3.1.2 Drawings

Insert your subsection material and for instance a few drawings...

The schematic illustrated in Fig.6.3 can represent some sort of algorithm.

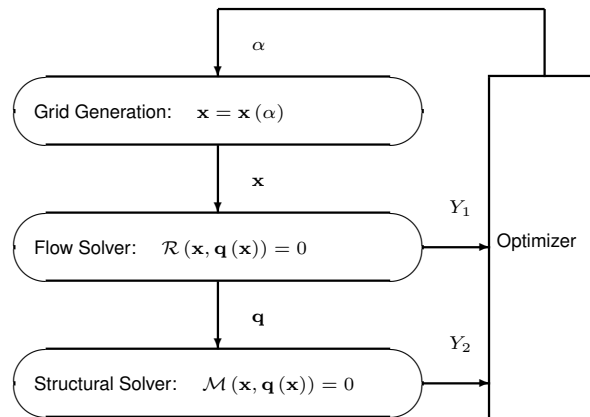


Figure 6.3: Schematic of some algorithm.

6.3.2 Equations

Equations can be inserted in different ways.

The simplest way is in a separate line like this

$$\frac{dq_{ijk}}{dt} + \mathcal{R}_{ijk}(\mathbf{q}) = 0. \quad (6.1)$$

If the equation is to be embedded in the text. One can do it like this $\partial \mathcal{R} / \partial \mathbf{q} = 0$.

It may also be split in different lines like this

$$\begin{aligned}
&\text{Minimize} && Y(\alpha, \mathbf{q}(\alpha)) \\
&\text{w.r.t.} && \alpha, \\
&\text{subject to} && \mathcal{R}(\alpha, \mathbf{q}(\alpha)) = 0 \\
&&& C(\alpha, \mathbf{q}(\alpha)) = 0.
\end{aligned} \tag{6.2}$$

It is also possible to use subequations. Equations (6.3a), (6.3b) and (6.3c) form the Navier–Stokes equations (6.3).

$$\frac{\partial \rho}{\partial t} + \frac{\partial}{\partial x_j} (\rho u_j) = 0, \tag{6.3a}$$

$$\frac{\partial}{\partial t} (\rho u_i) + \frac{\partial}{\partial x_j} (\rho u_i u_j + p \delta_{ij} - \tau_{ji}) = 0, \quad i = 1, 2, 3, \tag{6.3b}$$

$$\frac{\partial}{\partial t} (\rho E) + \frac{\partial}{\partial x_j} (\rho E u_j + p u_j - u_i \tau_{ij} + q_j) = 0. \tag{6.3c}$$

6.3.3 Tables

Insert your subsection material and for instance a few tables...

Make sure all tables presented are referenced in the text!

The caption should appear above the table.

Follow some guidelines when making tables:

- Avoid vertical lines
- Avoid “boxing up” cells, usually 3 horizontal lines are enough: above, below, and after heading
- Avoid double horizontal lines
- Add enough space between rows

Table 6.1: Table caption.

Model	C_L	C_D	C_{My}
Euler	0.083	0.021	-0.110
Navier–Stokes	0.078	0.023	-0.101

Make reference to Tab.6.1.

Tables 6.2 and 6.3 are examples of tables with merging columns:

An example with merging rows can be seen in Tab.6.4.

If the table has too many columns, it can be scaled to fit the text width, as in Tab.6.5.

Table 6.2: Memory usage comparison (in MB).

	Virtual memory [MB]	
	Euler	Navier–Stokes
Wing only	1,000	2,000
Aircraft	5,000	10,000
(ratio)	5.0×	5.0×

Table 6.3: Another table caption.

	$w = 2$			$w = 4$		
	$t = 0$	$t = 1$	$t = 2$	$t = 0$	$t = 1$	$t = 2$
$dir = 1$						
c	0.07	0.16	0.29	0.36	0.71	3.18
c	-0.86	50.04	5.93	-9.07	29.09	46.21
c	14.27	-50.96	-14.27	12.22	-63.54	-381.09
$dir = 0$						
c	0.03	1.24	0.21	0.35	-0.27	2.14
c	-17.90	-37.11	8.85	-30.73	-9.59	-3.00
c	105.55	23.11	-94.73	100.24	41.27	-25.73

Table 6.4: Yet another table caption.

ABC	header			
	1.1	2.2	3.3	4.4
IJK	group	0.5		0.6
		0.7		1.2

Table 6.5: Very wide table.

Variable	a	b	c	d	e	f	g	h	i	j
Test 1	10,000	20,000	30,000	40,000	50,000	60,000	70,000	80,000	90,000	100,000
Test 2	20,000	40,000	60,000	80,000	100,000	120,000	140,000	160,000	180,000	200,000

6.3.4 Mixing

If necessary, a figure and a table can be put side-by-side as in Fig.6.4

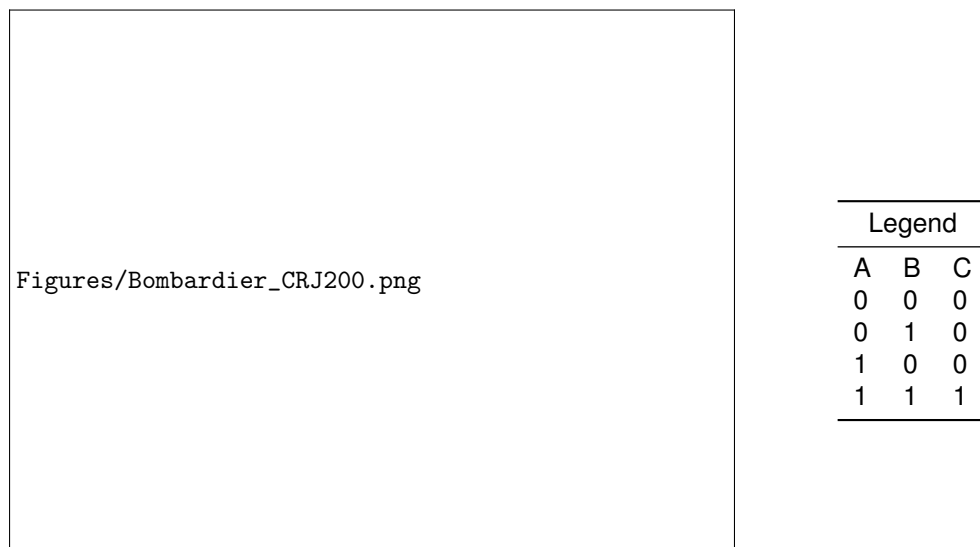


Figure 6.4: Figure and table side-by-side.

Chapter 7

Conclusions

Insert your chapter material here...

7.1 Achievements

The major achievements of the present work...

7.2 Future Work

A few ideas for future work...

Bibliography

- [1] K. Yoshida, B. Wilcox, et al. *Handbook of Robotics*. Springer Nature, second edition, 2016.
- [2] W. Burgard, M. Hebert, and M. Bennewitz. *World Modeling*. 2016.
- [3] P. Fankhauser, M. Bloesch, C. Gehring, M. Hutter, and R. Siegwart. Robot-centric elevation mapping with uncertainty estimates. In *Mobile Service Robotics*, pages 433–440. World Scientific, 2014.
- [4] R. Correal and G. Pajares. Onboard autonomous navigation architecture for a planetary surface exploration rover and functional validation using open-source tools. In *ESA Intl. Conf. on Advanced Space Technologies in Robotics and Automation*.
- [5] D. E. Smith, M. T. Zuber, H. V. Frey, J. B. Garvin, J. W. Head, D. O. Muhleman, G. H. Pettengill, R. J. Phillips, S. C. Solomon, H. J. Zwally, et al. Mars orbiter laser altimeter: Experiment summary after the first year of global mapping of mars. *Journal of Geophysical Research: Planets*, 2001.
- [6] P. G. Jayasekara, G. Ishigami, and T. Kubota. Testing and validation of autonomous navigation for a planetary exploration rover using opensource simulation tools. In *International Symposium on Artificial Intelligence, Robotics and Automation in Space*.
- [7] D. Amorim and R. Ventura. Towards efficient path planning of a mobile robot on rough terrain. In *2014 IEEE International Conference on Autonomous Robot Systems and Competitions (ICARSC)*.
- [8] Y. Yokokohji, S. Chaen, and T. Yoshikawa. Evaluation of traversability of wheeled mobile robots on uneven terrains by a fractal terrain model. *Advanced Robotics*, 2007.
- [9] G. Ishigami, K. Nagatani, and K. Yoshida. Path planning for planetary exploration rovers and its evaluation based on wheel slip dynamics. In *Proceedings 2007 IEEE International Conference on Robotics and Automation*.
- [10] R. Ventura and A. Ahmad. Towards optimal robot navigation in domestic spaces. *Lecture Notes in Artificial Intelligence (Subseries of Lecture Notes in Computer Science)*, 8992:318–331, 1 2015.
- [11] C. A. Brooks and K. Iagnemma. Self-supervised terrain classification for planetary surface exploration rovers. *Journal of Field Robotics*, 2012.

- [12] P. Fankhauser and M. Hutter. A Universal Grid Map Library: Implementation and Use Case for Rough Terrain Navigation. In A. Koubaa, editor, *Robot Operating System (ROS) – The Complete Reference (Volume 1)*, chapter 5. Springer, 2016. ISBN 978-3-319-26052-5. doi: 10.1007/978-3-319-26054-9{_}5. URL <http://www.springer.com/de/book/9783319260525>.
- [13] T. Foote and M. Purvis. Standard units of measure and coordinate conventions, 10 2010. URL <https://ros.org/reps/rep-0103.html#coordinate-frame-conventions>.
- [14] K. Kozłowski and D. Pazderski. Modeling and control of a 4-wheel skid-steering mobile robot. *International Journal of Applied Mathematics and Computer Science*, 14:477–496, 2004.
- [15] K. Iagnemma, C. Brooks, and S. Dubowsky. Visual, tactile, and vibration-based terrain analysis for planetary rovers. volume 2, pages 841–848, 2004.
- [16] K. Iagnemma and S. Dubowsky. Traction control of wheeled robotic vehicles in rough terrain with application to planetary rovers. *The International Journal of Robotics Research*, 23:1029–1040, 11 2004.
- [17] A. Peiret, E. Karpman, et al. Modelling of off-road wheeled vehicles for real-time dynamic simulation. *Journal of terramechanics*, pages 45–58, 4 2021.
- [18] G. Genta. *Introduction to the Mechanics of Space Robots*. Springer, 2012.
- [19] J. A. Sethian. Fast marching methods. *SIAM Review*, 41:199–235, 1999.
- [20] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl. CasADi – A software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, 2018.
- [21] A. Wächter and L. T. Biegler. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical programming*, 106:25–57, 2006.
- [22] C. M. Roithmayr and D. H. Hodges. *Dynamics: Theory and Application of Kane’s Method*. Cambridge University Press, 2015.

Appendix A

Mathematical snippet

A.1 Euler angles

Euler rotation angles defining an intrinsic $z - y - x$ (yaw-pitch-roll) rotation, with a rotation sequence of $z - y' - x''$ around the moving frame $\mathcal{B}$ is denoted as,

$$\mathbf{R}_{\mathcal{B}}^{\mathcal{W}} = \mathbf{R}_z(\psi)\mathbf{R}_y(\theta)\mathbf{R}_x(\phi) \quad (\text{A.1})$$

, with the previous multiplication to yield a mapping from frame $\mathcal{B}$ to $\mathcal{W}$, which stands as,

$$\begin{aligned} \mathbf{R}_{\mathcal{B}}^{\mathcal{W}}(\phi, \theta, \psi) &= \begin{bmatrix} \cos(\psi) & -\sin(\psi) & 0 \\ \sin(\psi) & \cos(\psi) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos(\theta) & 0 & \sin(\theta) \\ 0 & 1 & 0 \\ -\sin(\theta) & 0 & \cos(\theta) \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\phi) & -\sin(\phi) \\ 0 & \sin(\phi) & \cos(\phi) \end{bmatrix} \\ &= \begin{bmatrix} \cos(\psi)\cos(\theta) & \cos(\psi)\sin(\theta)\sin(\phi) - \sin(\psi)\cos(\phi) & \cos(\psi)\sin(\theta)\cos(\phi) + \sin(\psi)\sin(\phi) \\ \sin(\psi)\cos(\theta) & \sin(\psi)\sin(\theta)\sin(\phi) + \cos(\psi)\cos(\phi) & \sin(\psi)\sin(\theta)\cos(\phi) - \cos(\psi)\sin(\phi) \\ -\sin(\theta) & \cos(\theta)\sin(\phi) & \cos(\theta)\cos(\phi) \end{bmatrix} \end{aligned} \quad (\text{A.2})$$

Rotation matrices are singular ($\det(\mathbf{R}_{\mathcal{B}}^{\mathcal{W}}) = 1$) and symmetric, $(\mathbf{R}_{\mathcal{B}}^{\mathcal{W}})^T = (\mathbf{R}_{\mathcal{B}}^{\mathcal{W}})^{-1} = \mathbf{R}_{\mathcal{W}}^{\mathcal{B}}$. Euler rotation matrices can be derived from 24 different conventions ranging from extrinsic or intrinsic rotation with different rotation orders. More information on this can be found in [22].

A.2 Skew-symmetric matrix

A skew-symmetric matrix S is a square matrix whose transpose equals its negative, $S^T = -S$. In this thesis, three by three matrices are employed which take the following form,

$$S(\mathbf{r} = (x, y, z)) = \begin{bmatrix} 0 & -z & y \\ z & 0 & -x \\ -y & x & 0 \end{bmatrix} \quad (\text{A.3})$$

In case this matrix is multiplied by a column vector $\mathbf{a}$, it is the same as applying the cross product between $\mathbf{r}$ and $\mathbf{a}$. Then, it is stated that,

$$\mathbf{S}(\mathbf{r})\mathbf{a} = \mathbf{r} \times \mathbf{a} \quad (\text{A.4})$$

A.3 Inertia tensor

The inertia tensor of a rigid body is given by,

$$\mathbf{I} = \begin{bmatrix} I_{xx} & I_{xy} & I_{xz} \\ I_{yx} & I_{yy} & I_{yz} \\ I_{zx} & I_{zy} & I_{zz} \end{bmatrix} \quad (\text{A.5})$$

, where the products of inertia are equal between each other ($I_{xy} = I_{yx}$, $I_{xz} = I_{zx}$, $I_{yz} = I_{zy}$).

A.4 Differentiation of a vector

If a vector is expressed with respect to a non-inertial frame $\mathcal{P}$, it is still possible to differentiate that same vector with respect to an inertial frame $\mathcal{Q}$ [?].

Let $\mathbf{p} = p_1 \mathbf{i} + p_2 \mathbf{j} + p_3 \mathbf{k}$, with $\mathbf{i}$, $\mathbf{j}$, $\mathbf{k}$ to be the canonical vectors of a non-inertial frame $\mathcal{P}$. Then, the differentiation of $\mathbf{p}$ with respect to $\mathcal{Q}$ is given by,

$$\left. \frac{d\mathbf{p}}{dt} \right|_{\mathcal{Q}} = \dot{p}_1 \mathbf{i} + \dot{p}_2 \mathbf{j} + \dot{p}_3 \mathbf{k} + p_1 \frac{d\mathbf{i}}{dt} + p_2 \frac{d\mathbf{j}}{dt} + p_3 \frac{d\mathbf{k}}{dt} \quad (\text{A.6})$$

, where $\left. \frac{d\mathbf{p}}{dt} \right|_{\mathcal{P}} = \dot{p}_1 \mathbf{i} + \dot{p}_2 \mathbf{j} + \dot{p}_3 \mathbf{k}$ because the canonical vectors $\mathbf{i}$, $\mathbf{j}$, $\mathbf{k}$ do not change with respect to time if the observer is located at the origin of $\mathcal{P}$. According to the general expression $\frac{d\mathbf{r}}{dt} = \boldsymbol{\Omega} \times \mathbf{r}$, it comes that $\frac{d\mathbf{i}}{dt} = \boldsymbol{\Omega} \times \mathbf{i}$, $\frac{d\mathbf{j}}{dt} = \boldsymbol{\Omega} \times \mathbf{j}$, $\frac{d\mathbf{k}}{dt} = \boldsymbol{\Omega} \times \mathbf{k}$. Then, equation A.6 may be rewritten as,

$$\left. \frac{d\mathbf{p}}{dt} \right|_{\mathcal{Q}} = \left. \frac{d\mathbf{p}}{dt} \right|_{\mathcal{P}} + \boldsymbol{\Omega} \times \mathbf{p} \quad (\text{A.7})$$

, where $\boldsymbol{\Omega}$ denotes the angular velocity of frame $\mathcal{P}$ with respect to frame $\mathcal{Q}$.

A.5 Parallel-axis theorem

Appendix B

Miscellaneous

B.1 Elevation maps

Below, the depiction of the elevation maps, which are used as test-beds for the control systems, are rendered.



Figure B.1: Elevation maps

B.2 Traversability maps

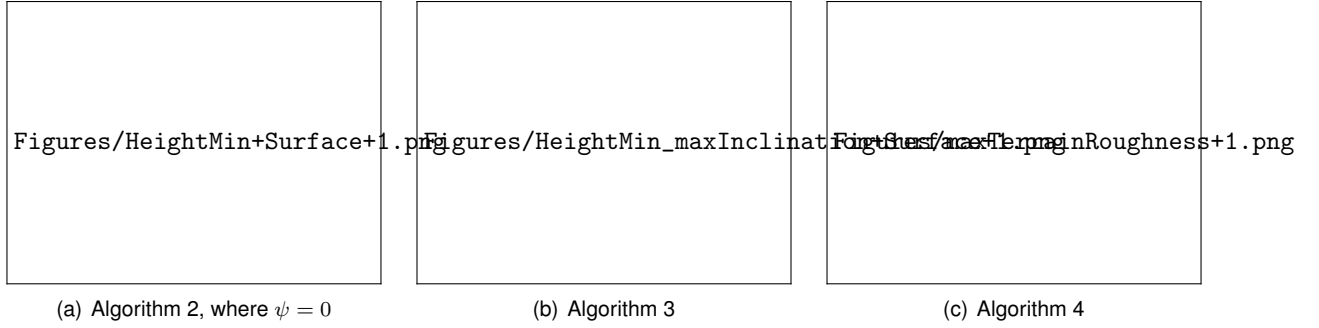


Figure B.2: Traversability maps (case 2; map 1)

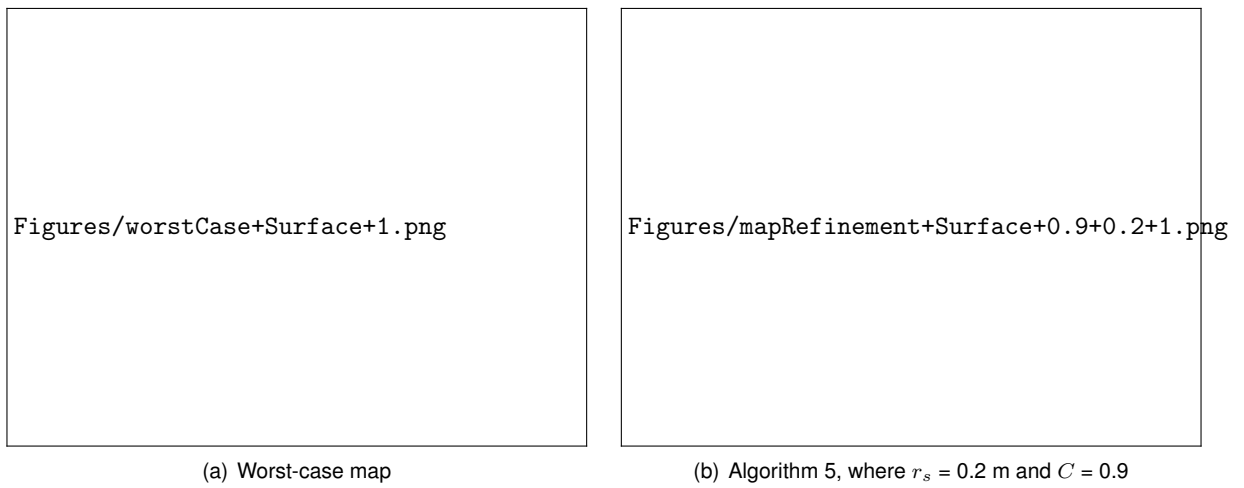


Figure B.3: Traversability maps (case 2; map 1)